

Apuntes - Clase 9

Segmentación

Sistemas Operativos

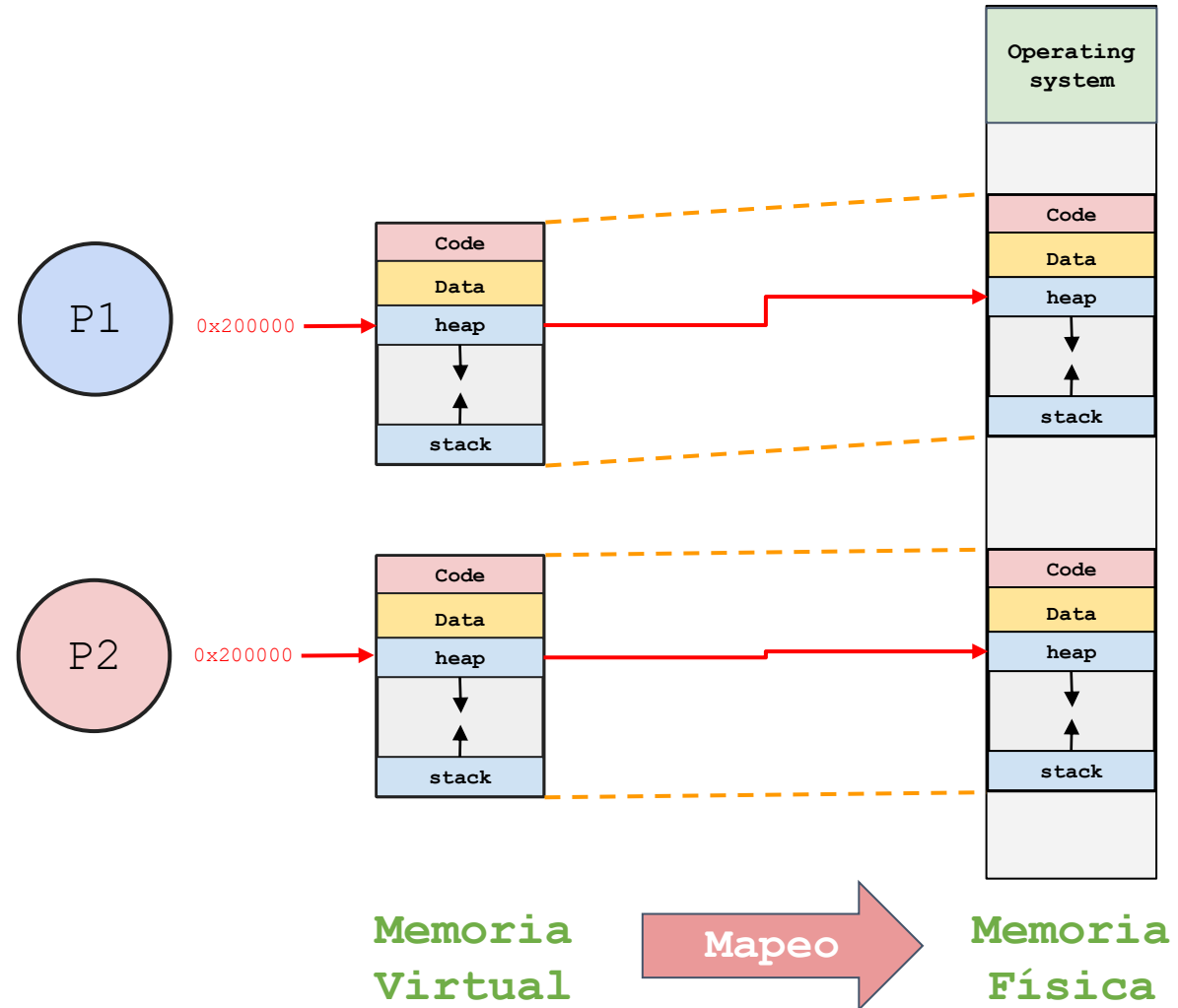
Universidad de Antioquia

Facultad Ingeniería

Ingeniería de Sistemas

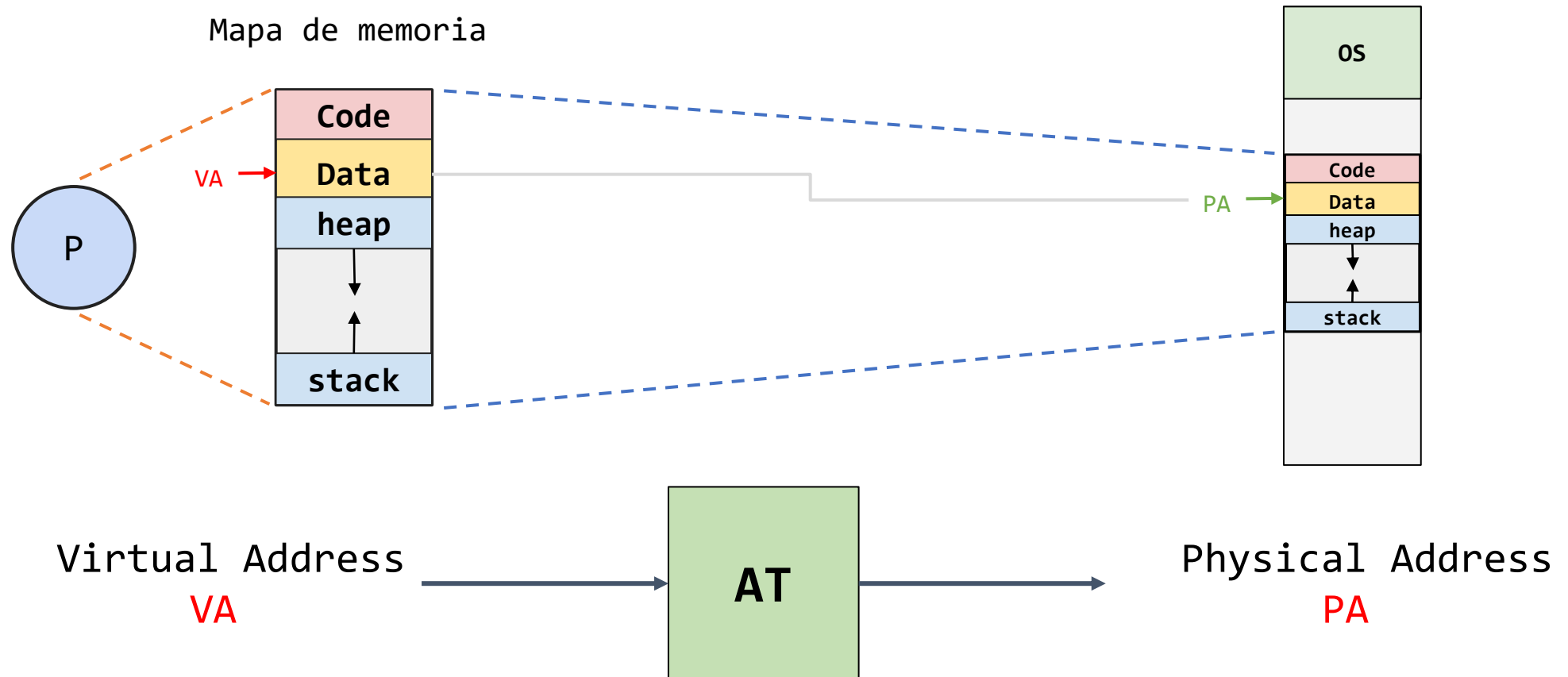
9/10/2026

Memoria Física – Memoria Virtual



Address translation - Mapeo

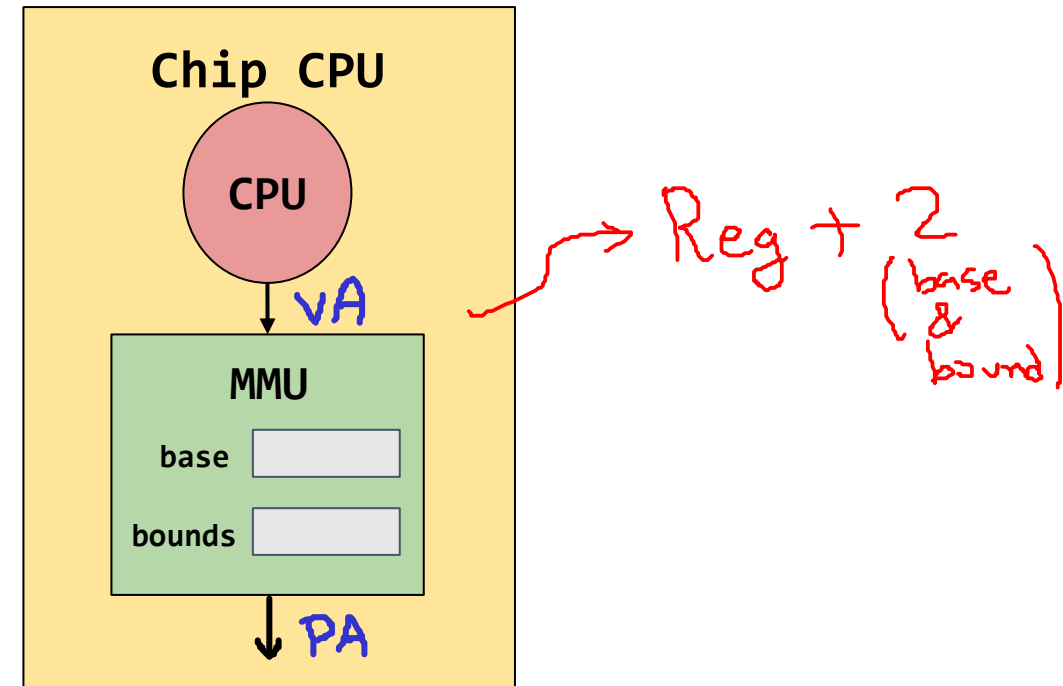
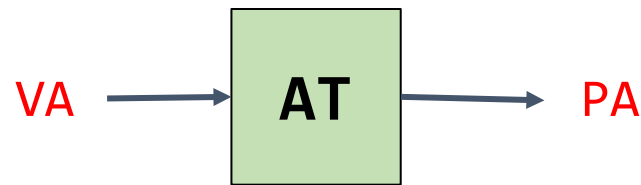
Asociación entre direcciones virtuales y físicas



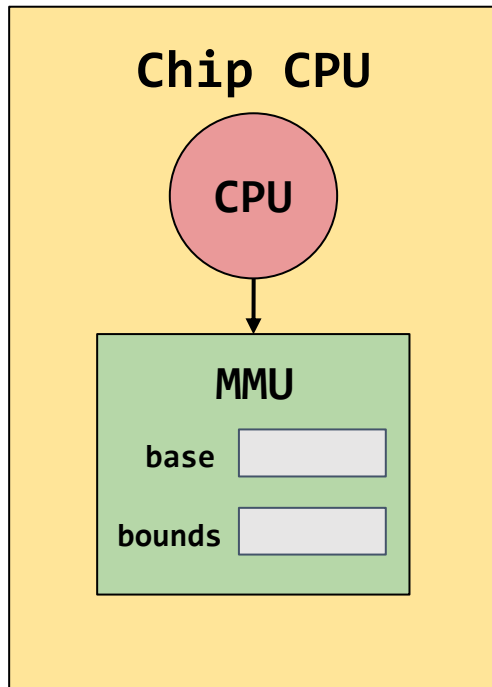
Dynamic Relocation (base & bounds)

Registros asociados:

- **base**: Almacena la dirección física (**offset**) a partir de la cual el address space de un proceso será mapeado.
- **bounds**: Almacena el tamaño del address space.



Traducción de direcciones usando Dynamic Relocation

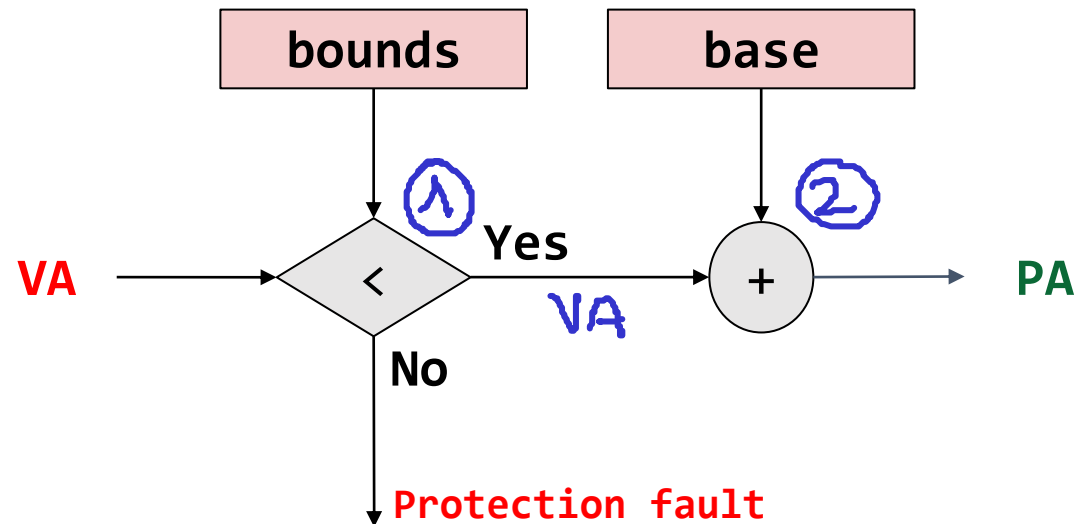


1 $0 \leq VA < \text{bounds}$

Check Limits

2 $PA = VA + \text{base}$

Calculo de la dirección Física (PA)



Ejemplo 1 - Clase anterior

Un proceso con un espacio de direccionamiento de 4KB fue asignado a una memoria física de 16KB. Teniendo en cuenta lo anterior, realizar la traducción de direcciones virtuales a físicas mostradas en la siguiente tabla:

Datos:

- base = 16 KB
- bounds = 4KB

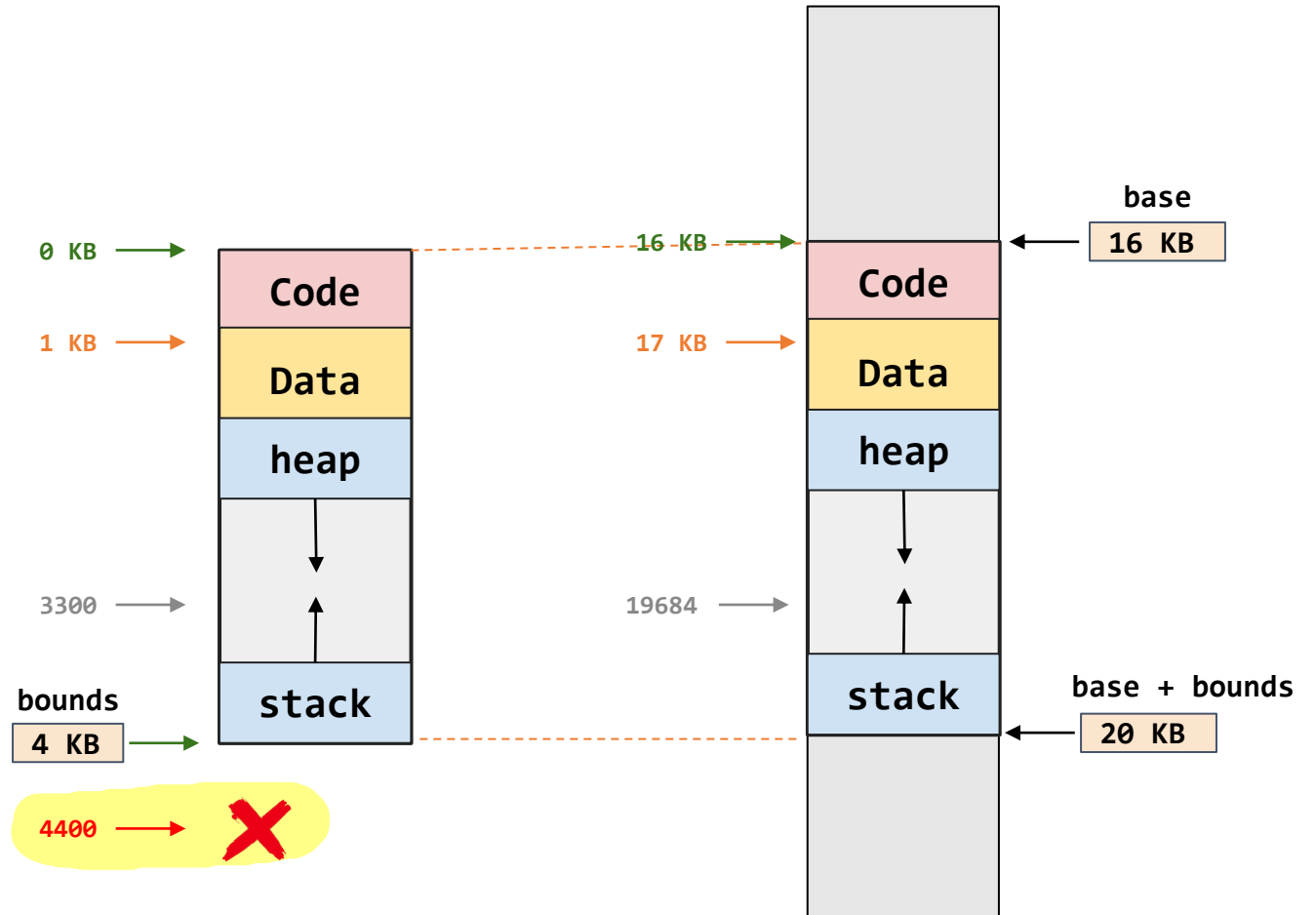
Virtual Address (VA)	$0 \leq VA < 4KB$	$PA = VA + 16KB$
0	OK	$0 + 16KB = 16KB$
1 KB	OK	$1 + 16KB = 17KB$
3300	OK	$3300 + 16KB = 3300 + 16(1024)=19684$
4400	Error	Dirección fuera de rango

Ejemplo 1 - Clase anterior

Datos:

- base = 16 KB
- bounds = 4KB

Virtual Address (VA)	Physical Address (PA)
0	16KB
1 KB	17KB
3300	19684
4400	Dirección fuera de rango



Ejemplo 2

Suponga que se tienen dos procesos con un espacio de direcciones (Address Space) de 1KB. Estos procesos se mapean en memoria física (Physical Address) a partir de las direcciones 1 KB y 4KB respectivamente.

- Dibuje un bosquejo de cómo queda la memoria física.
- Si el proceso P1 ejecuta la instrucción **load 100, R** ¿A qué dirección de memoria accediendo para obtener el valor que será llevado a R?
- Si el proceso P2 ejecuta la instrucción **load 100, R** ¿A qué dirección de memoria accediendo para obtener el valor que será llevado a R?
- Analice la ejecución **load 100, R** para el proceso 1

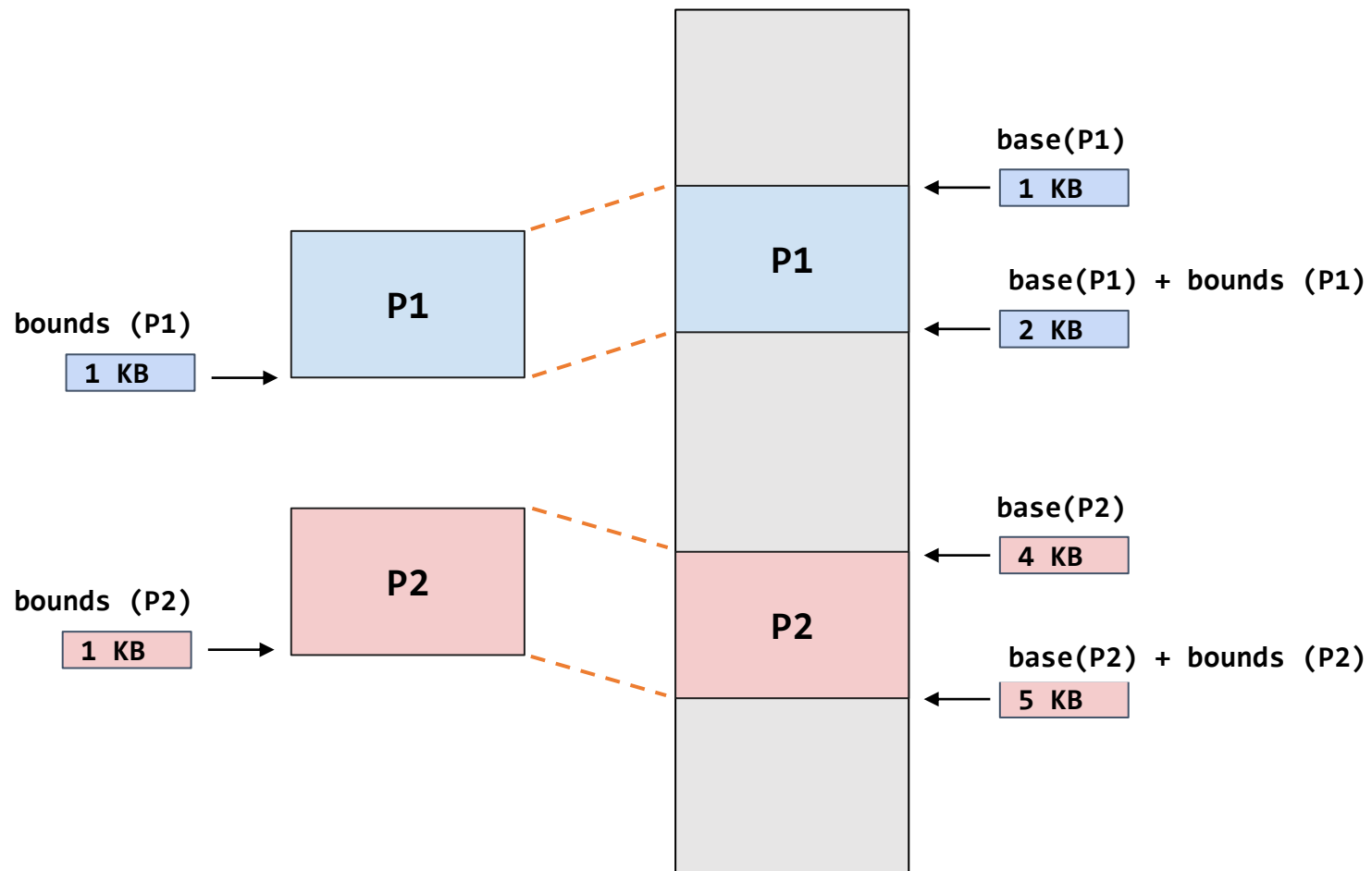
Ejemplo 2

Suponga que se tienen dos procesos con un espacio de direcciones (Address Space) de 1KB. Estos procesos se mapean en memoria física (Physical Address) a partir de las direcciones 1 KB y 4KB respectivamente.

Ejemplo 2

Datos:

- $\text{base}(P1) = 1 \text{ KB}$
- $\text{bounds}(P1) = 1 \text{ KB}$
- $\text{base}(P2) = 4 \text{ KB}$
- $\text{bounds}(P2) = 1 \text{ KB}$



Ejemplo 2

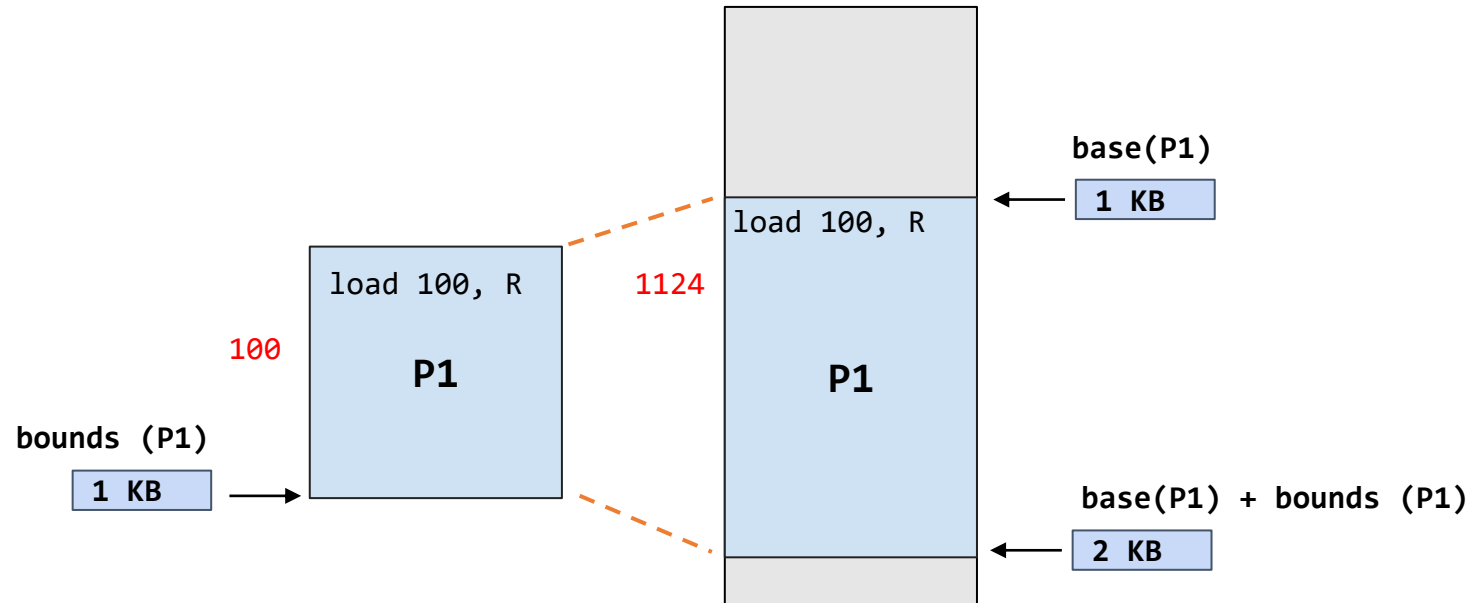
Datos:

- $\text{base}(P1) = 1 \text{ KB}$
- $\text{bounds}(P1) = 1 \text{ KB}$
- $VA = 100$
- $PA(VA=100) = ?$

Instrucción: `load 100, R`

$$PA(P1) = VA + \text{base}(P1)$$

$$PA(P1) = 100 + 1K = 100 + 1024 = \mathbf{1124}$$



$$0 \leq VA < \text{bounds}$$

$$PA = VA + \text{base}$$

Ejemplo 2

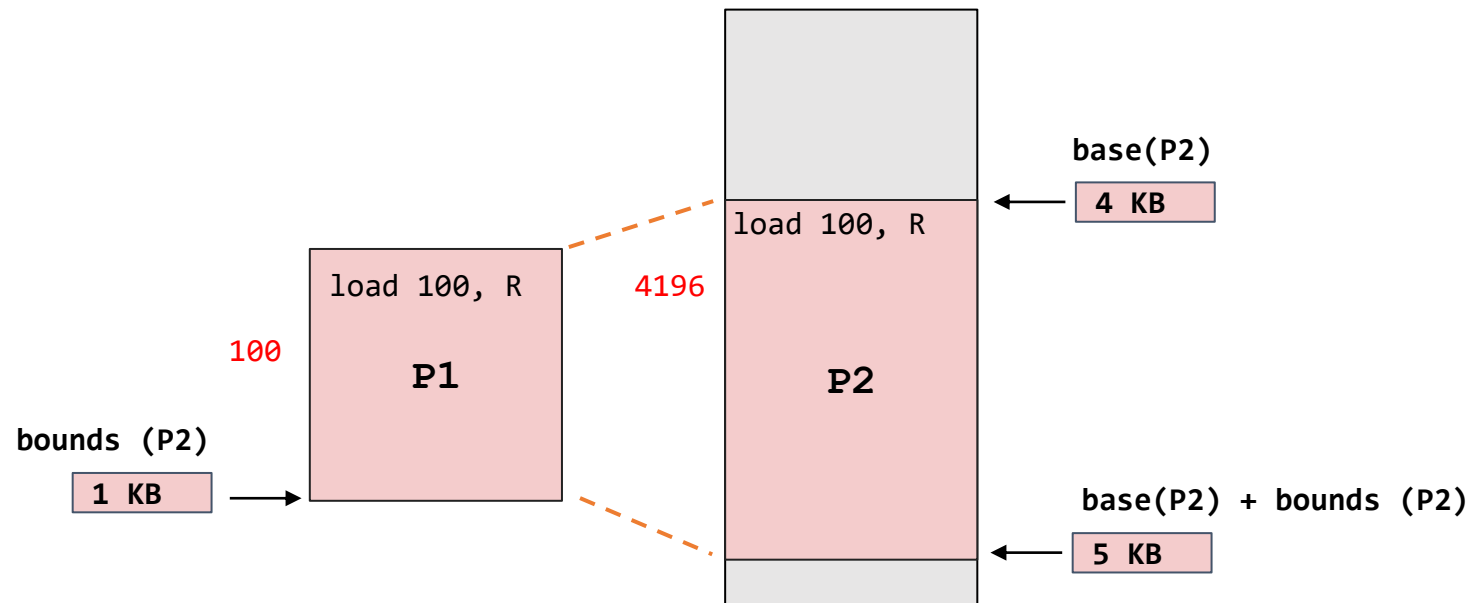
Datos:

- $\text{base}(\text{P2}) = 4 \text{ KB}$
- $\text{bounds}(\text{P2}) = 1 \text{ KB}$
- $\text{VA} = 100$
- $\text{PA}(\text{VA}=100) = ?$

Instrucción: load 100, R

$$\text{PA}(\text{P2}) = \text{VA} + \text{base}(\text{P2})$$

$$\text{PA}(\text{P2}) = 100 + 1\text{K} = 100 + 4096 = 4196$$



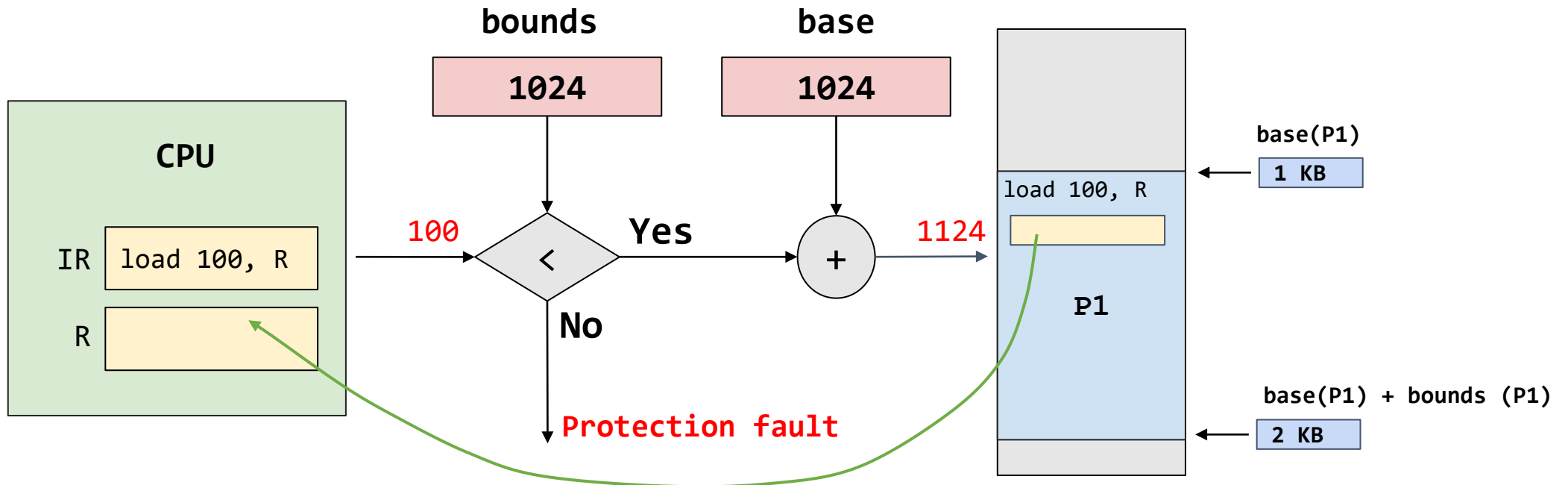
$$\text{PA} = \text{VA} + \text{base}$$

$$0 \leq \text{VA} < \text{bounds}$$

Ejemplo 3

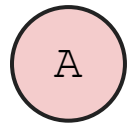
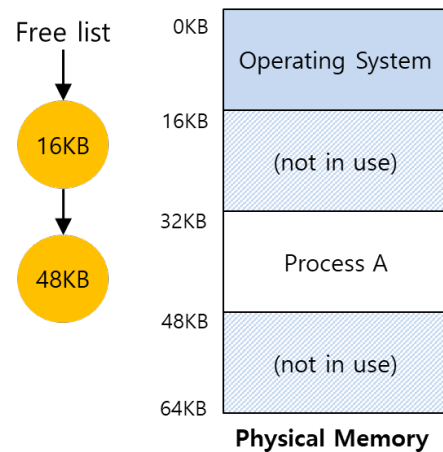
Análisis de la ejecución de la instrucción **load 100, R** para el proceso **P1**

1. Conversión de dirección lógica a física para **P1**: $PA(VA = 100) = 1124$
2. Se lanza el load a **PA = 1124**
3. Se pone el resultado en **R**.

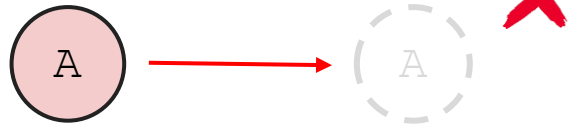
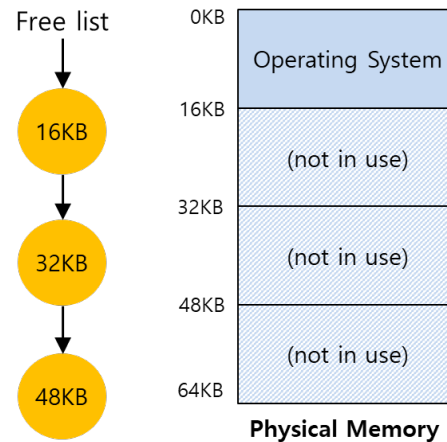


Acciones del sistema operativo

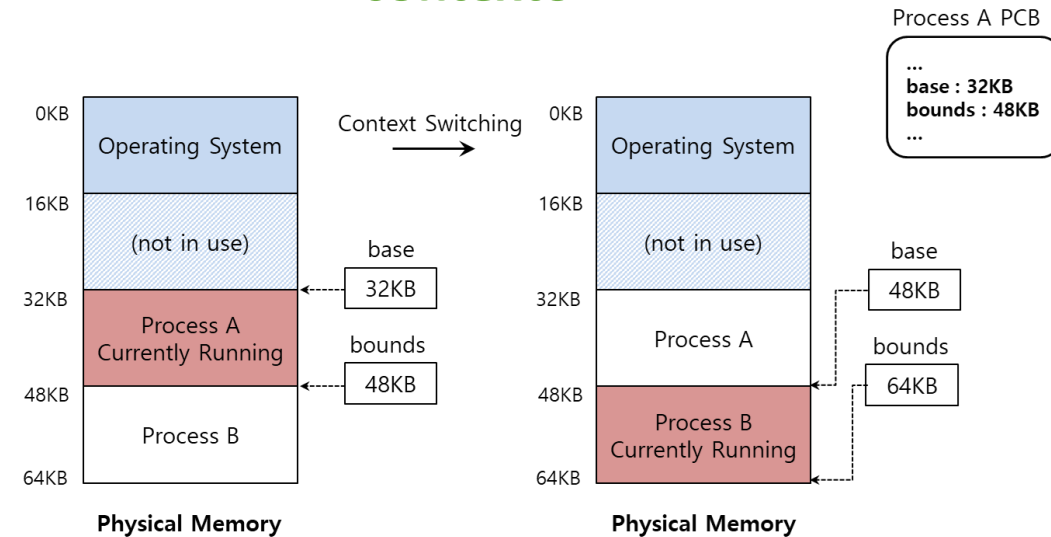
Creación de un proceso



Terminación de un proceso



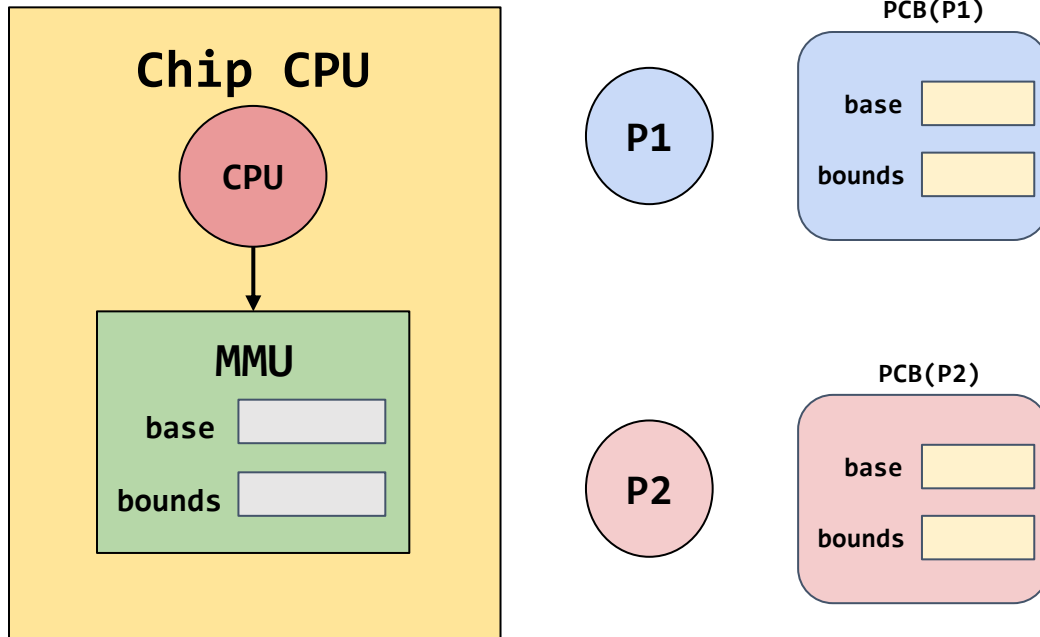
Cambio de contexto



Ventajas y desventajas de la reasignación dinámica (base & bound)

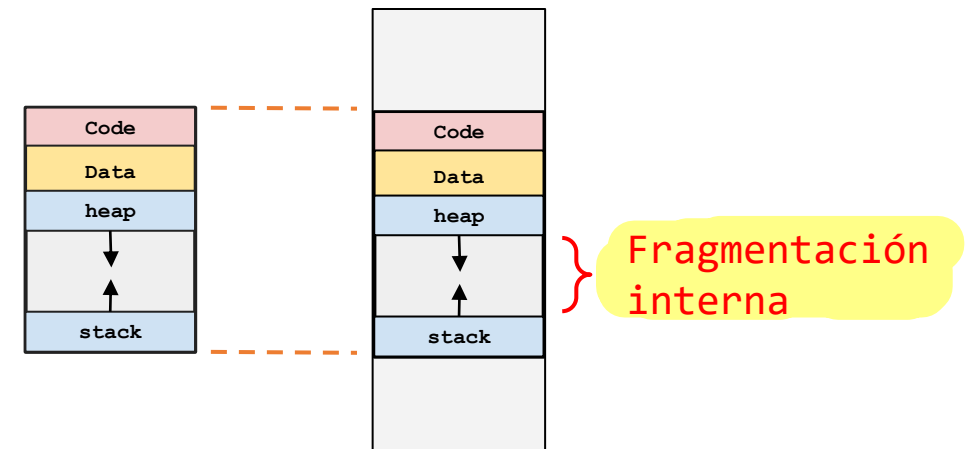
Ventajas

- Rápida y simple.
- Ofrece protección
- Poco overhead (2 registros por proceso)



Desventajas

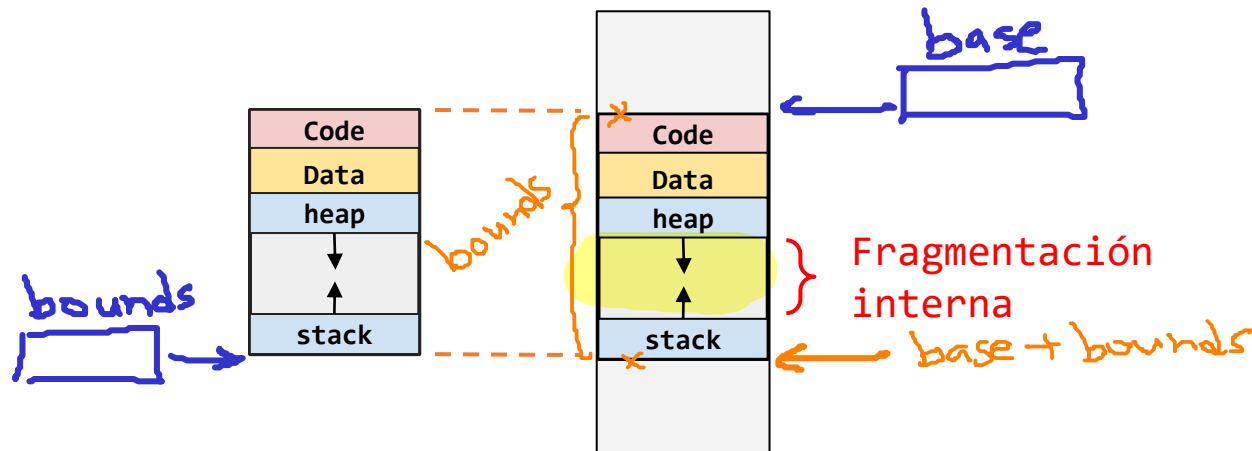
- No es flexible.
- Gasto de memoria (sobre todo para espacios de direccionamiento grandes) → Fragmentación interna.



Pregunta clave

¿Como soportar grandes espacio de direcciones?

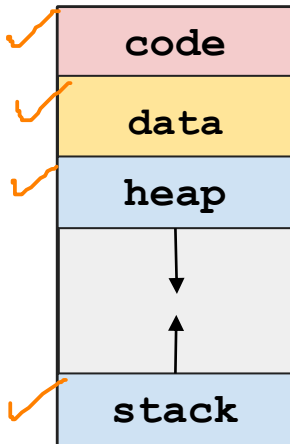
¿Cómo soportar un espacio de direcciones grande con un (potencial) espacio libre grande entre el **stack** y el **heap**?



Segmentación

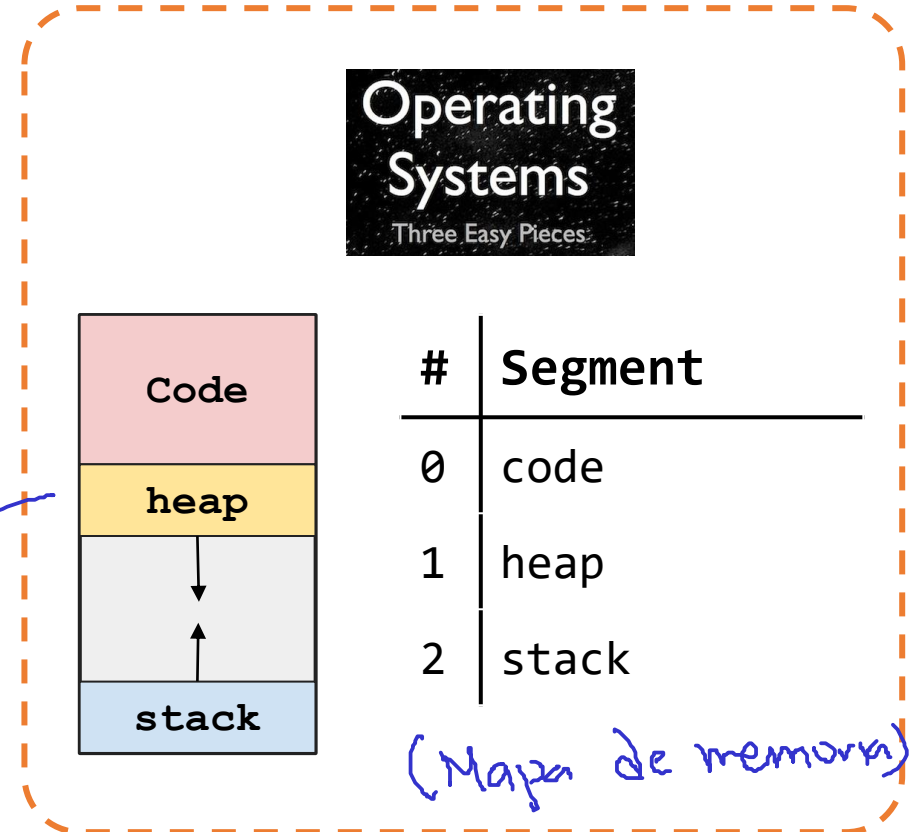
- **Segmento:** porción contigua del espacio de direccionamiento (con un tamaño particular).

4 segms.



#	Segment
0	Code
1	data
2	heap
3	stack

3 segms.

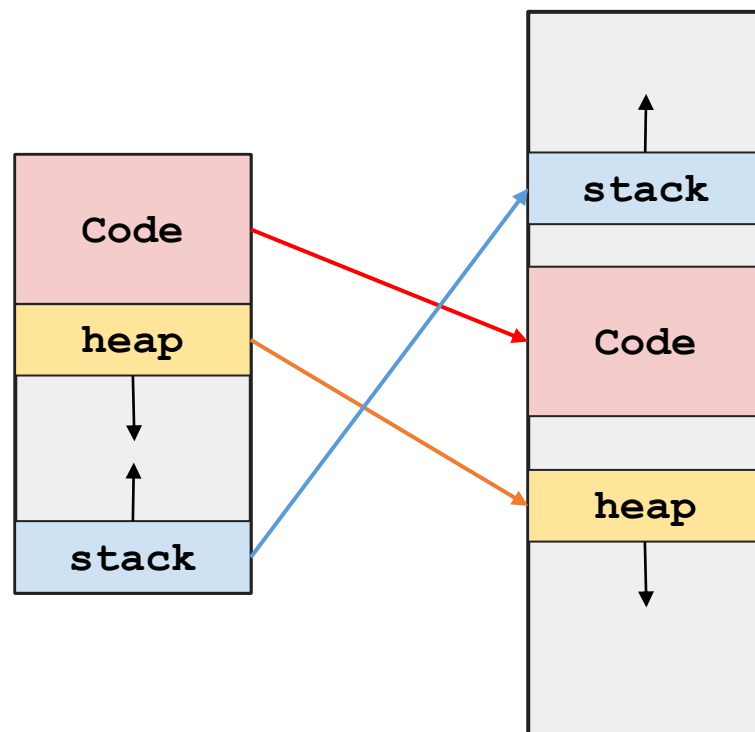


#	Segment
0	code
1	heap
2	stack

(Mapa de memoria)

Segmentación

- **Idea clave:** Cada segmento puede ser ubicado en una parte diferentes de la memoria física.

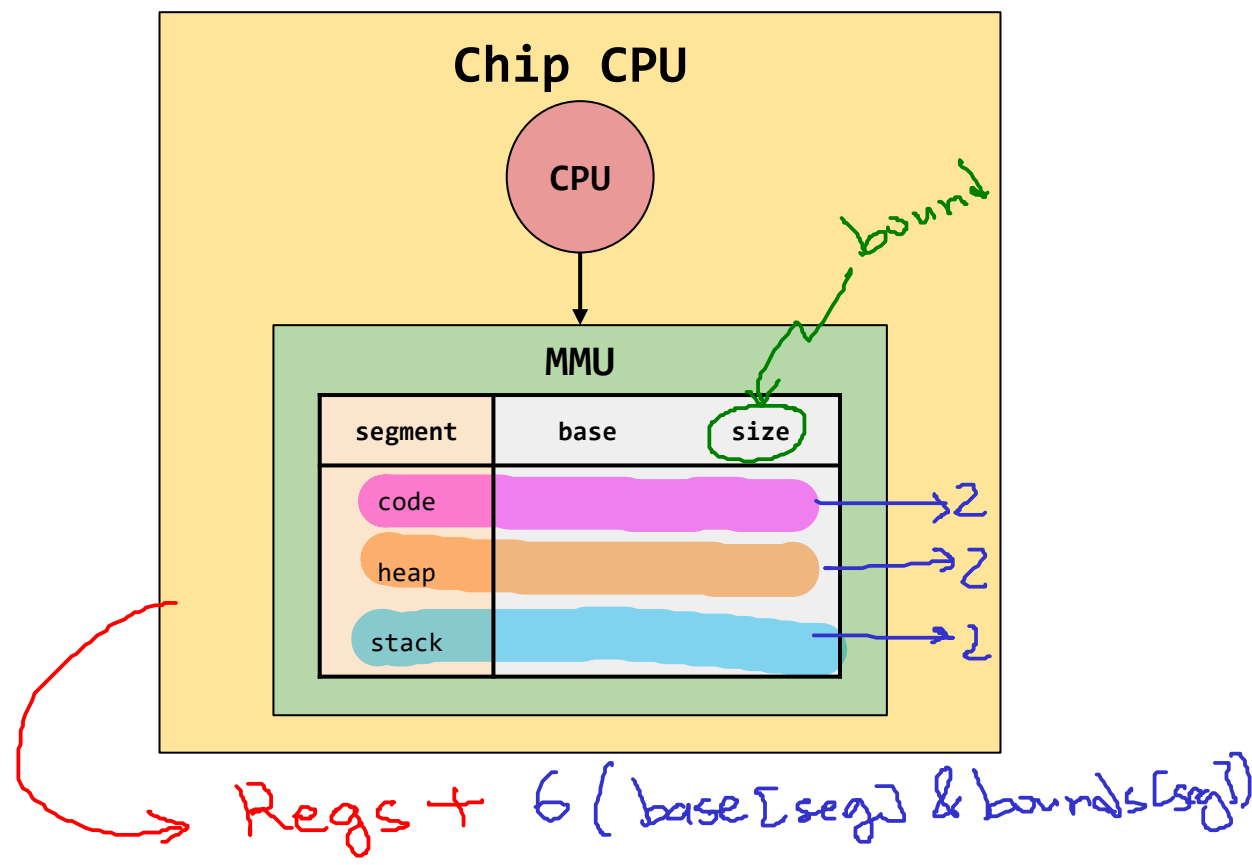
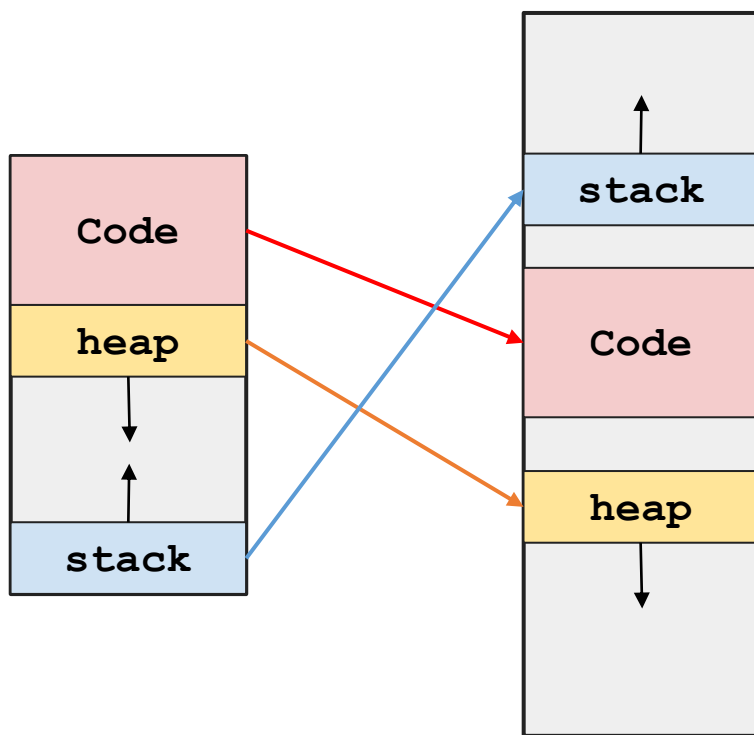


Cada segmento tiene su propio conjunto de registros para el mapeo

segment	base	size
code		
heap		
stack		

Segmentación

- Cada **segmento** puede ser ubicado en una parte diferentes de la memoria física tiene sus propios registros **base & bounds**.
- La MMU tendrá una tabla con tres pares de registros donde cada uno de estos pares estará relacionado a un segmento.

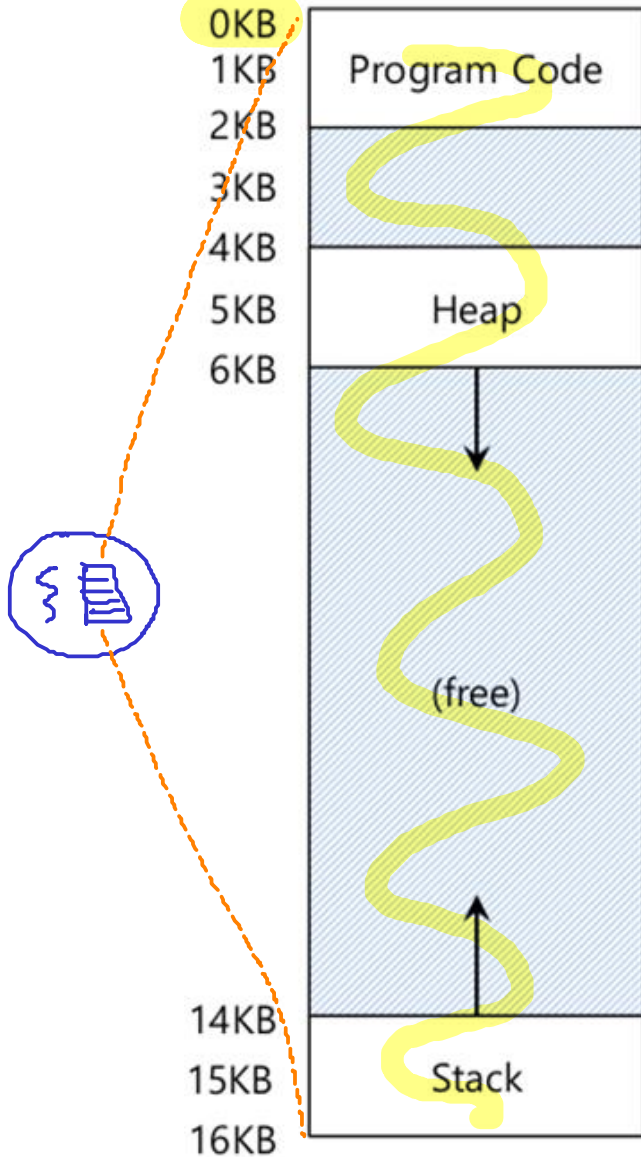


Traducción de direcciones (I)

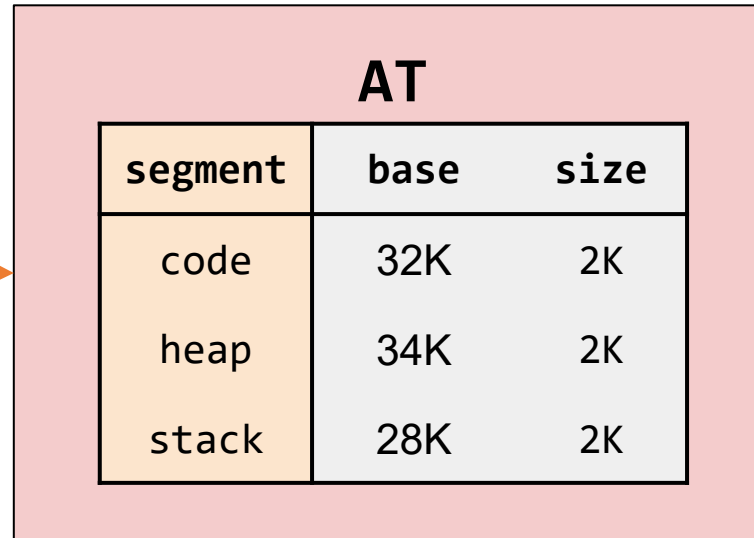
Address Space (AS)

Memoria Virtual

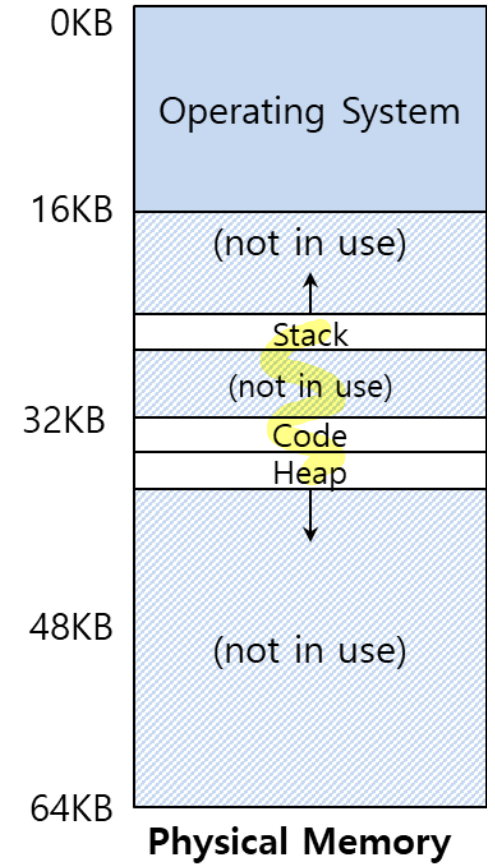
Physical Memory (PM)



VA



PA



Memoria virtual

Preguntas:

- ¿Cuántos bits son necesarios para direccionar la memoria física?
- ¿Cuál es el rango de direcciones? $[0, MAX) = [0, MAX-1]; MAX = 2^n$
- ¿Cuántos bits son necesarios para hacer referencia a cada uno de los segmentos?

Solución:

- size(AS) = 16K
- MAX = 16K
- n = ?
- Rango de direcciones = ?
- n(seg) = ?

① $n = ?$

$$\text{size(AS)} = 2^n$$

$$16K = 2^n$$

$$(2^4)(2^{10}) = 2^n$$

$$2^{14} = 2^n$$

$n = 14$ bits



② Rango:

$$[0, 2^n)$$

$$[0, 2^{14}) =$$

$$[0, 16384)$$

$$\text{min} = 0$$

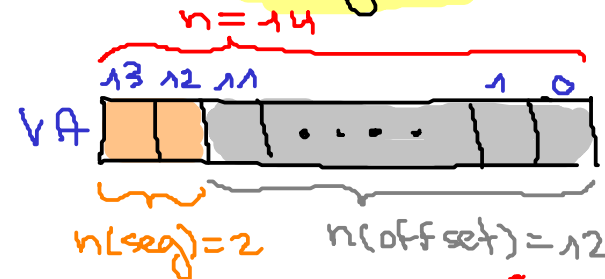
$$\text{max} = 16383$$

③ $n(\text{seg}) = ?$

3 segm {
code
heap
stack

$$2^2 = 4$$

$$n(\text{seg}) = 2$$



$$n(\text{seg}) = 2$$

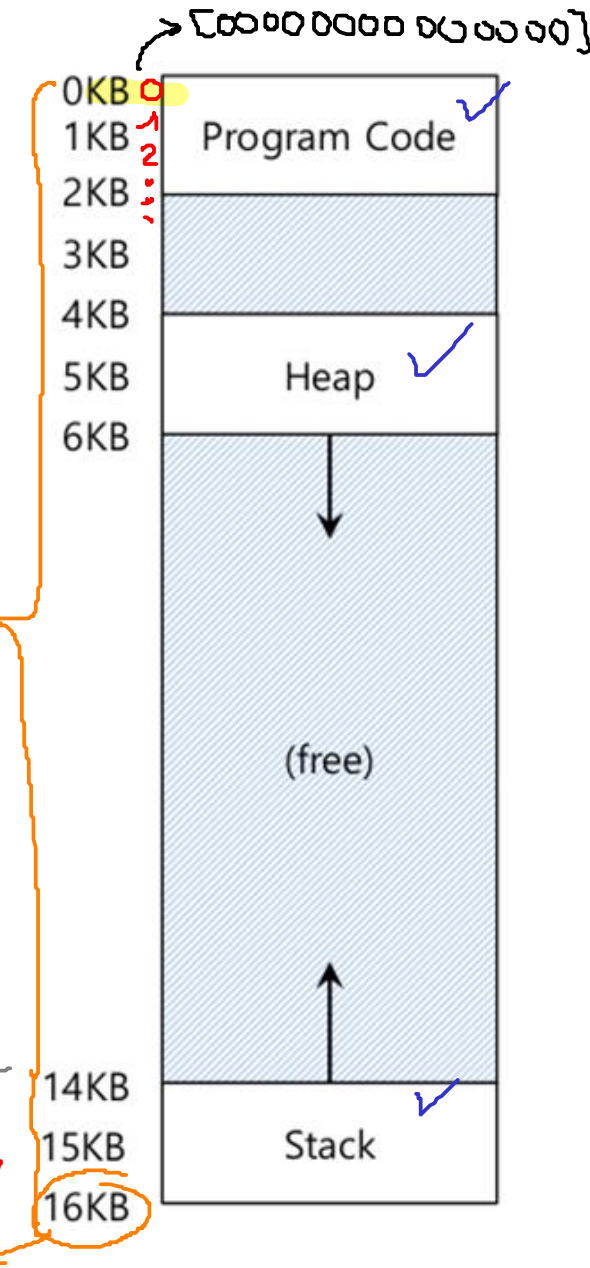
$$n(\text{offset}) = 12$$

$$\text{size(VM)} = 16K$$

$$16382$$

$$16383$$

$$[1111111111111111] \text{ MAX}$$



Memoria física

Preguntas:

- ¿Cuántos bits son necesarios para direccionar la memoria física? ($n=?$)
- ¿Cuál es el rango de direcciones? $[0, MAX] = [0, MAX-1] = [0, 2^n-1]$
- ¿Cuántos bits son necesarios para hacer referencia a cada uno de los segmentos? $n(seg)=?$

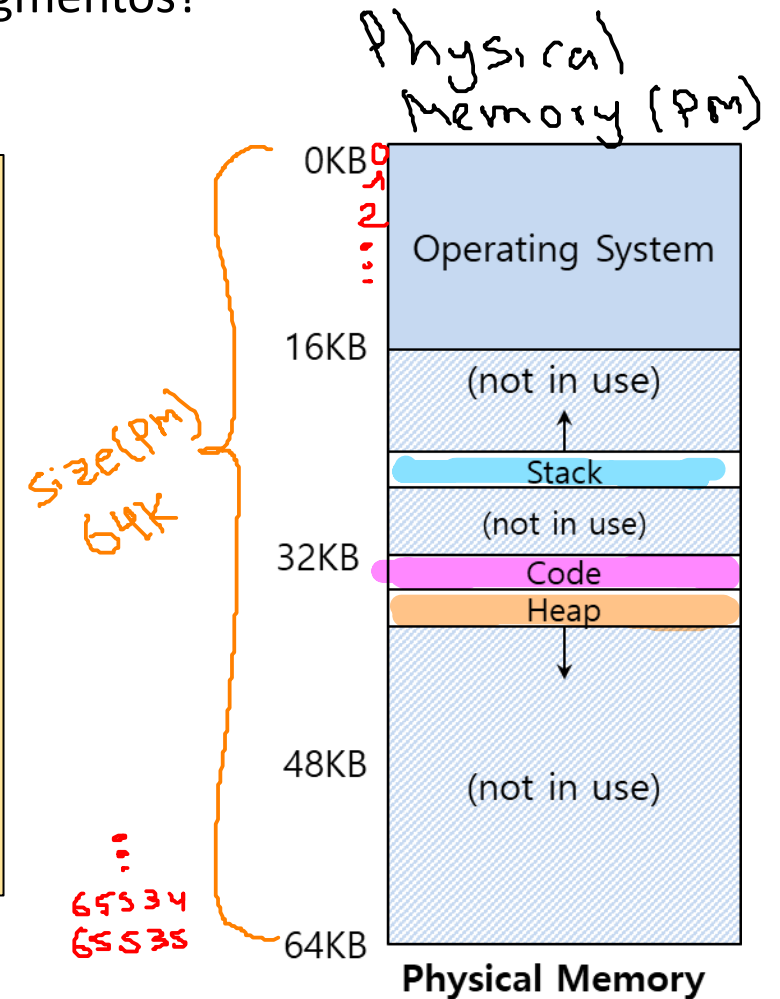
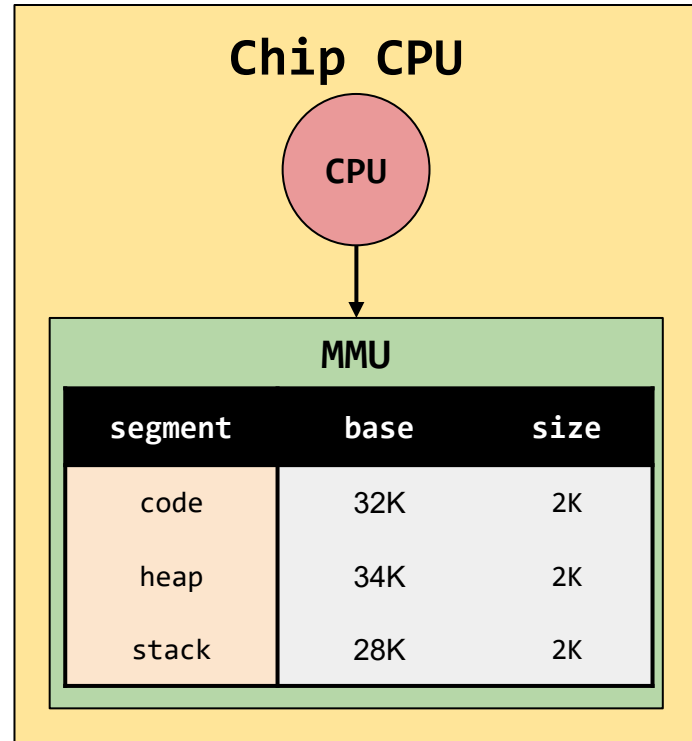
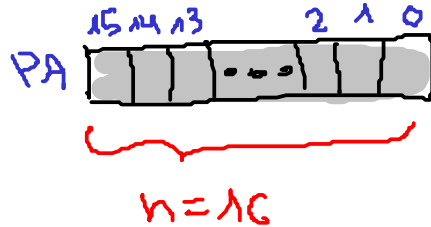
Datos:

- $size(PM) = 64K$
- $n=?$
- $n(seg)=?$

① $size(PM) = 2^n$
 $64K = 2^n$
 $(2^6)(2^{10}) = 2^n$
 $2^{16} = 2^n$
 $n = 16$

② $[0, 2^n - 1] =$
 $[0, 2^{16} - 1] =$
 $[0, 65536 - 1] =$
 $[0, 65535]$

③ $n(seg)=?$
 $seg = 3$
 $n(seg) = 2$



Traducción de direcciones (I)

Pregunta:

- Si $VA = 100$, ¿cual es el valor de PA ?

segment	base	size
code	32K	2K
heap	34K	2K
stack	28K	2K

Datos:

- $VA = 100$
- $PA = ?$

$VA \in \text{code} \rightarrow VA = \text{offset}$

$$\textcircled{1} \quad 0 \leq VA < \text{bounds}[\text{code}]$$

$\downarrow$
 $VA = \text{offset} = 100$

$$0 \leq \text{offset} < \text{bound}[\text{code}]$$

$$0 \leq 100 < 2K \quad \checkmark$$

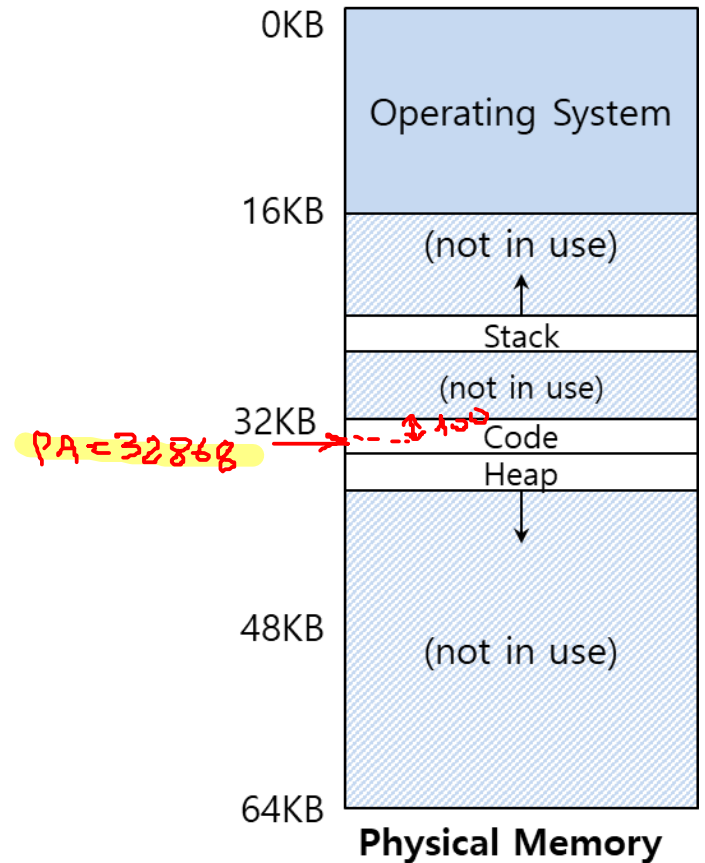
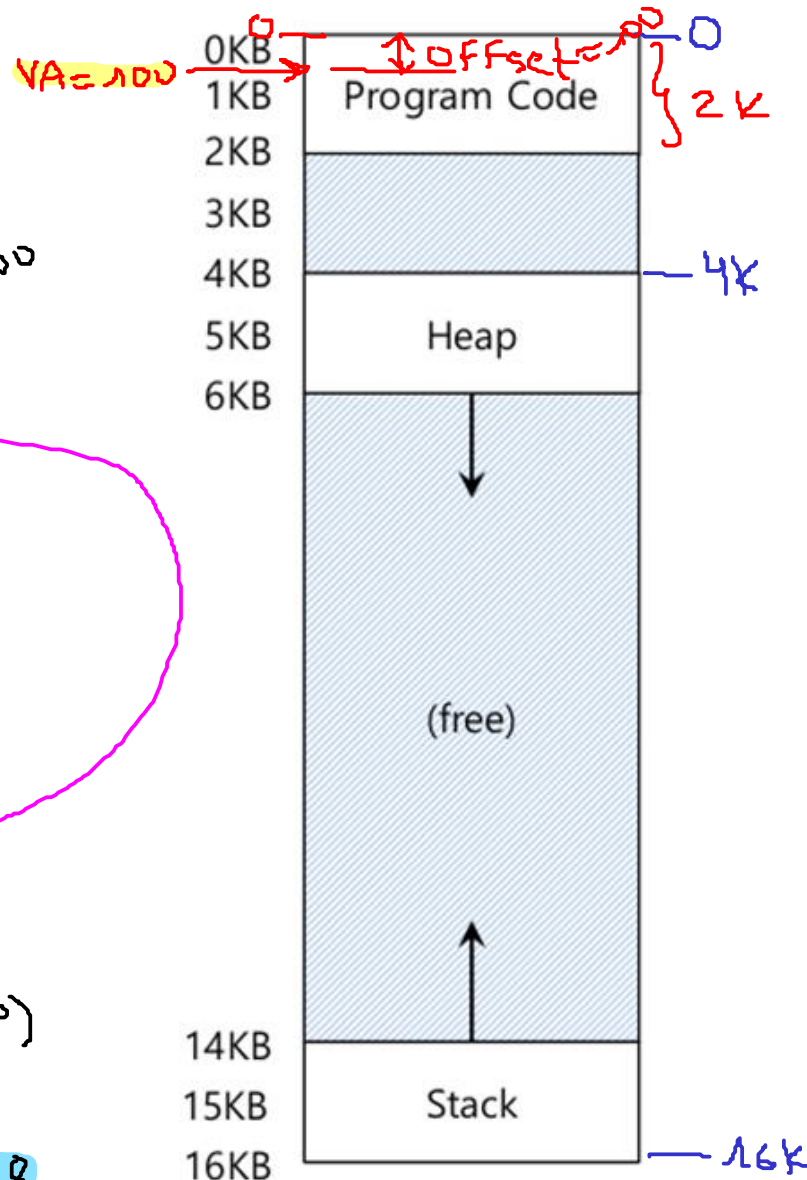
$$\textcircled{2} \quad PA = VA + \text{base}[\text{code}]$$

$\downarrow$
 $VA = \text{offset}$

$$PA = 100 + 32K = 100 + 32(2^{10})$$

$$PA = 32868$$

$$VA = 100 \xrightarrow{AT} PA = 32868$$



Traducción de direcciones (I)

Pregunta:

- Si $VA = 4200$, ¿cual es el valor de PA ?

segment	base	size
code	32K	2K
heap	34K	2K
stack	28K	2K

$VA = 4200$

$PA = ?$

$4096 = 4KB$
 $VA = 4200$

$VA \in \text{heap} \rightarrow \text{offset} = VA - VA(\text{heap})$

$$\text{offset} = VA - 4096 = 4200 - 4096 = 104$$

$\text{offset} = 104$

① $0 \leq \text{offset} < \text{size}(\text{heap})$

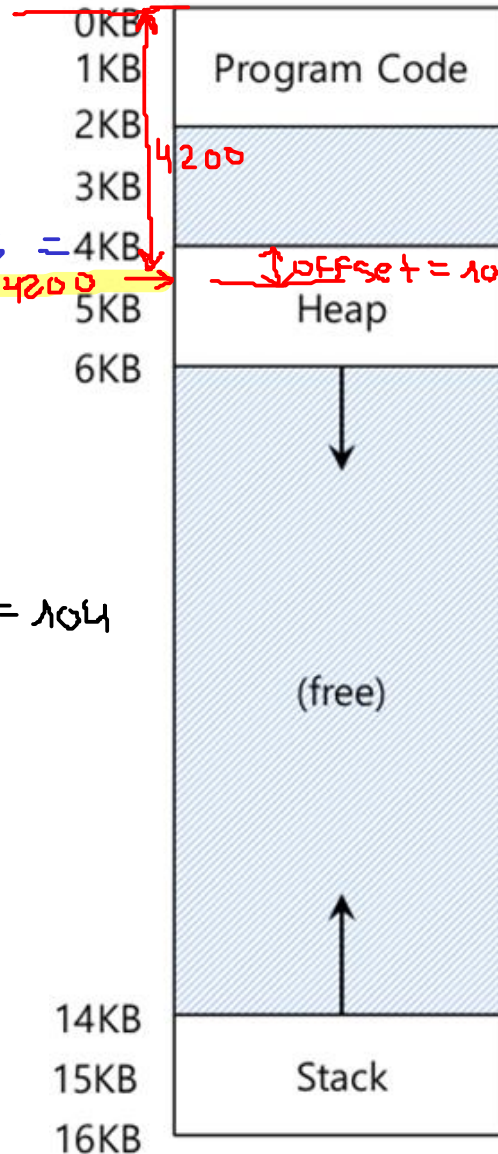
$\rightarrow 0 \leq 104 < 2K$ ✓

② $PA = \text{offset} + \text{base}(\text{heap})$

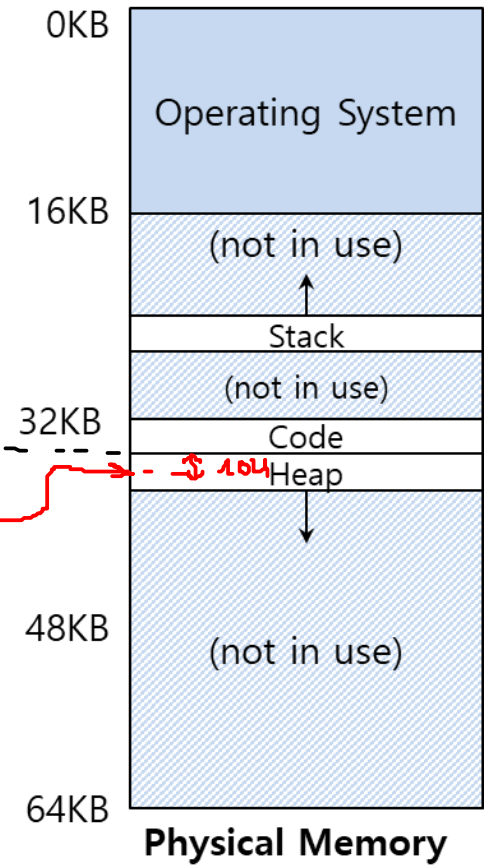
$$PA = 104 + 34K = 104 + 34(2^{10})$$

$PA = 34920$

$VA = 4200 \xrightarrow{AT} PA = 34920$



$VA(\text{base})$
 $4096 (0)$



Traducción de direcciones (I)

Pregunta:

- Si $VA = 7K$, ¿cual es el valor de PA ?

segment	base	size
code	32K	2K
heap	34K	2K
stack	28K	2K

$$VA = 7K = 7098$$

$$VA \in \text{heap} \rightarrow \text{offset} = VA - VA(\text{heap})$$

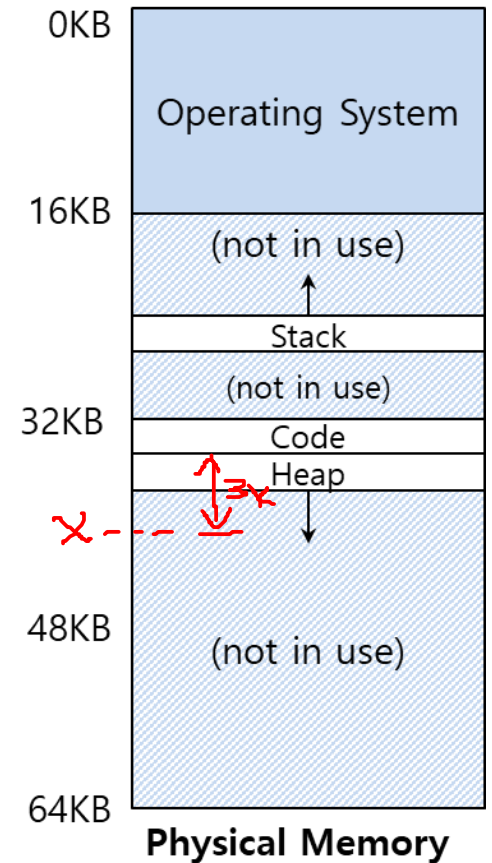
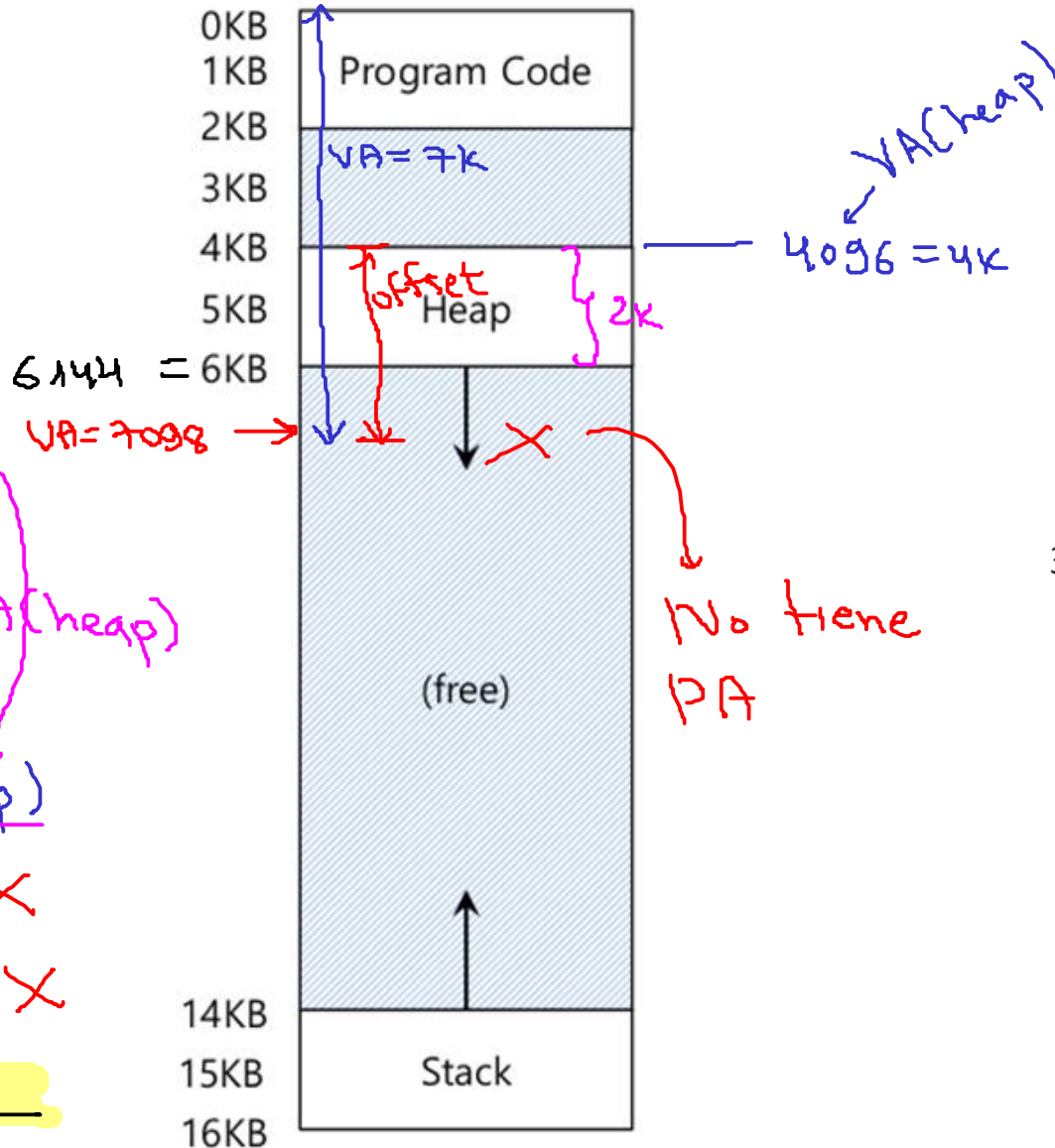
$$\text{offset} = 7K - 4K = 3K$$

$$\textcircled{1} \quad 0 \leq \text{offset} < \text{size}(\text{heap})$$

$$0 \leq 3K < 2K \quad \times$$

$$0 \leq 3072 < 2048 \quad \times$$

Seg. Fault $\rightarrow PA = \underline{\hspace{2cm}}$



Traducción de direcciones (I)

Segmento code

En este segmento se cumple que: **VA = offset** ✓

$$\text{offset} = \text{VA}$$

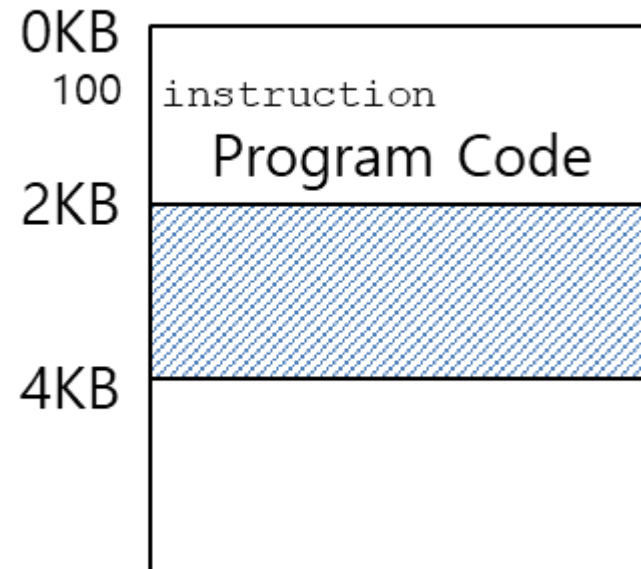
$$\text{PA} = \text{offset} + \text{base}(\text{code})$$

$$0 \leq \text{offset} < \text{size}(\text{code})$$

Donde:

- **offset**: offset segmento code.
- **base**: Valor del registro base del segmento code.
- **PA**: Physical Address.

segment	base	size
code	32K	2K
heap	34K	2K
stack	28K	2K



Traducción de direcciones (I)

Segmento code - Ejemplo ✓

1. Chequear límites

$$0 \leq \text{offset} < \text{size}(\text{code})$$

$$0 < 100 < 2K$$

2. Obtener la dirección física

$$\text{PA} = \text{offset} + \text{base}(\text{code})$$

$$\text{PA} = 100 + 32K$$

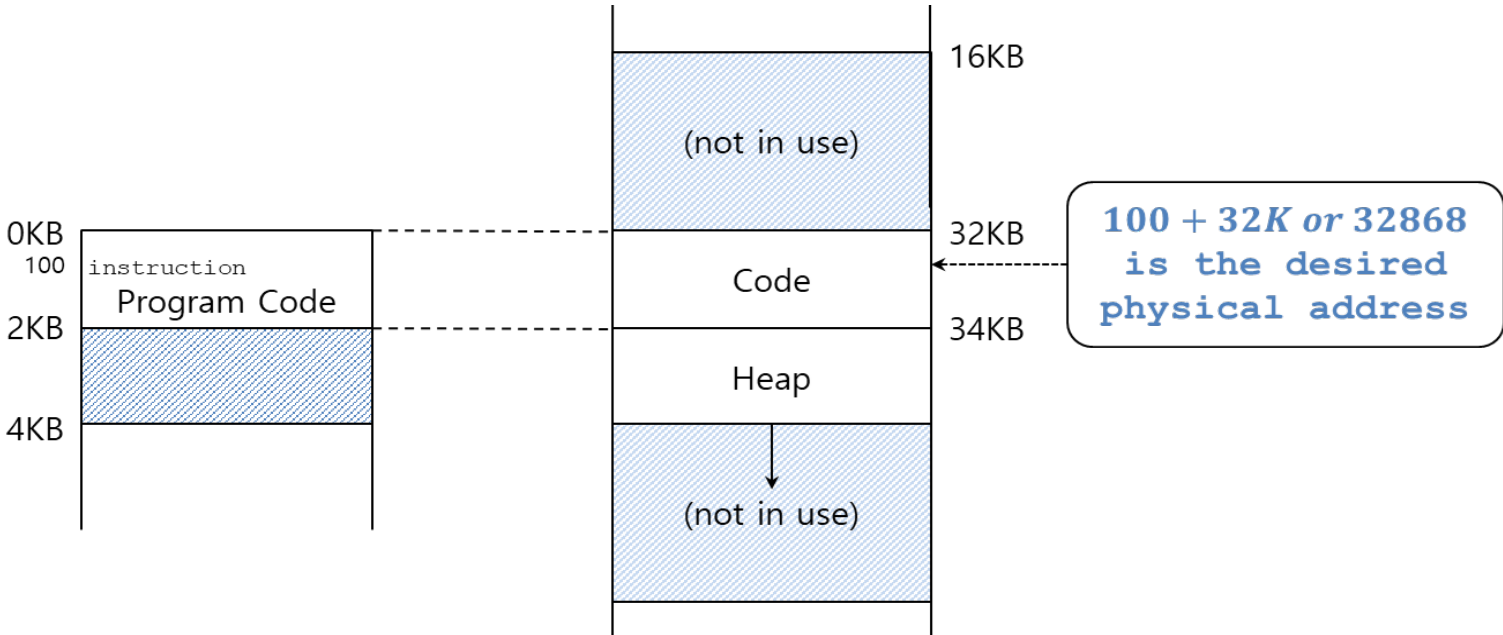
$$\text{PA} = 100 + 32(1024)$$

$$\text{PA} = 32868$$

Pregunta: ¿Cuál es la PA de VA = 100?

- VA ubicada en el code: offset = VA = 100

segment	base	size
code	32K	2K



Traducción de direcciones (I)

Segmento heap ✓

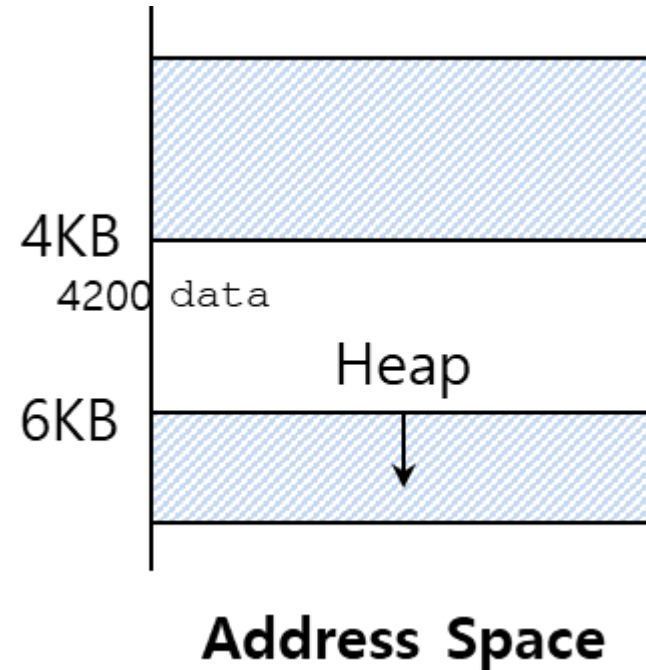
En este caso **VA + offset** no es la dirección física correcta

$$\text{offset} = \text{VA} - \text{VA}(\text{heap})$$

$$\text{PA} = \text{offset} + \text{base}(\text{heap})$$

$$0 \leq \text{offset} < \text{size}(\text{heap})$$

segment	base	size
code	32K	2K
heap	34K	2K
stack	28K	2K



Traducción de direcciones (I)

Segmento heap ✓

1. Obtener offset(heap)

$$\text{offset} = \text{VA} - \text{VA}(\text{heap})$$

$$\text{offset} = 4200 - 4\text{K}$$

$$\text{offset} = 4200 - 4(1024)$$

$$\text{offset} = 104$$

2. Verificar límites

$$0 \leq \text{offset} < \text{size}(\text{heap})$$

$$0 < 104 < 2\text{K}$$

3. Obtener PA

$$\text{PA} = \text{offset} + \text{base}(\text{heap})$$

$$\text{PA} = \text{offset} + \text{base}(\text{heap})$$

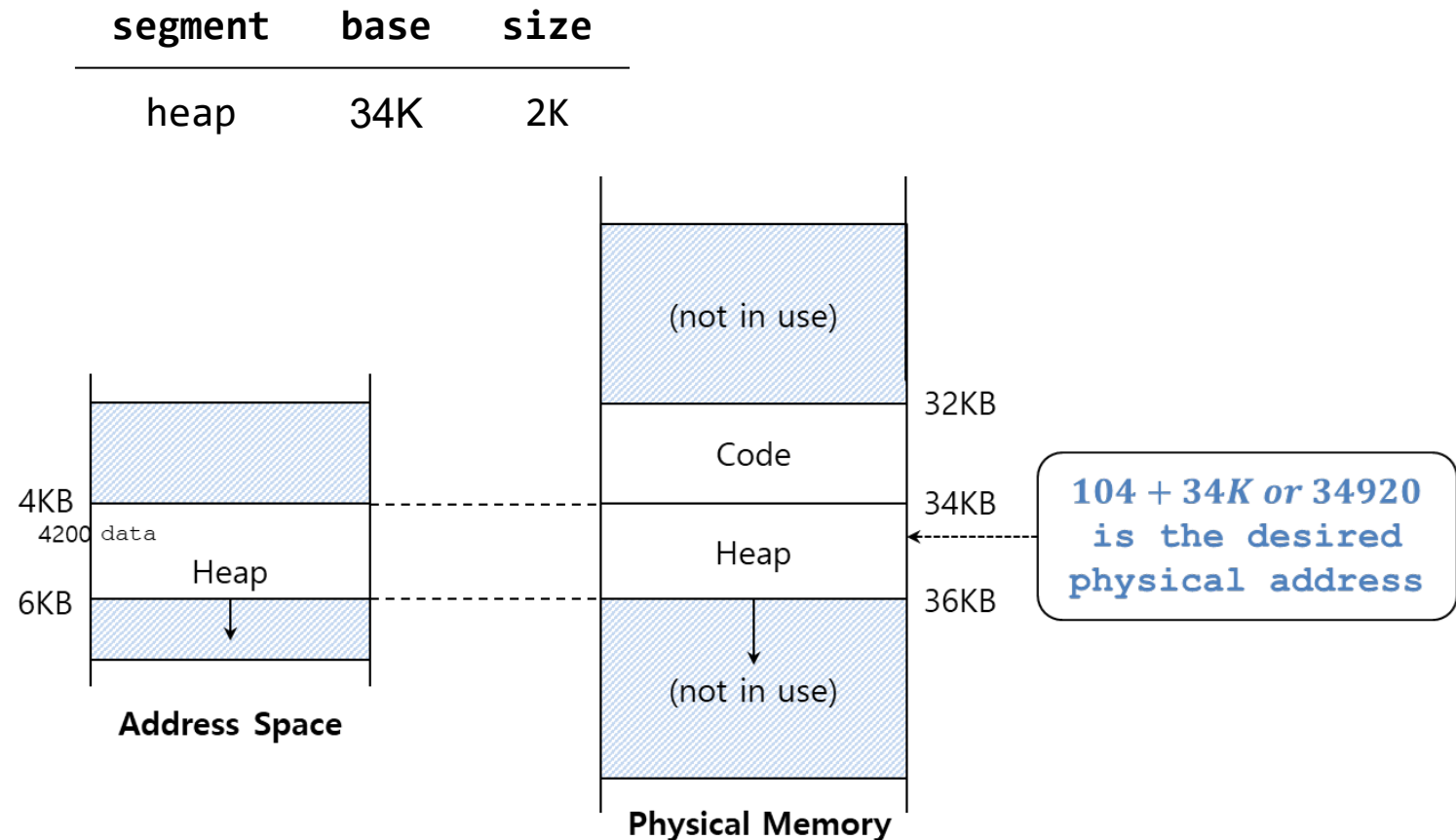
$$\text{PA} = 104 + 34\text{K}$$

$$\text{PA} = 104 + 34(1024)$$

$$\text{PA} = 34920$$

Pregunta: ¿Cuál es la PA de VA = 4200?

- Va ubicada en el heap: VA = 4200

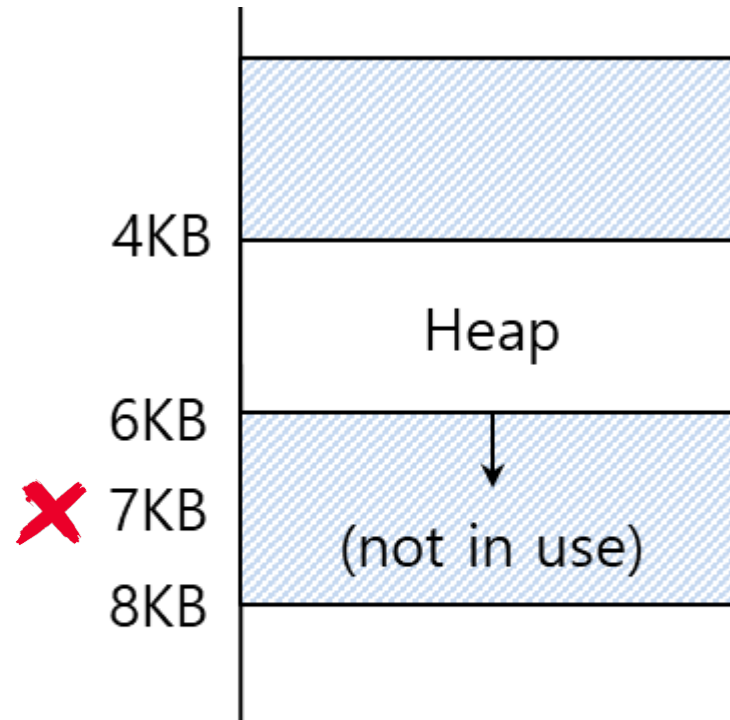


Traducción de direcciones (I)

Fallo de segmentación - segmentation fault

- Acceso a una **dirección ilegal** ✓

segment	base	size
code	32K	2K
heap	34K	2K
stack	28K	2K



Address Space

Traducción de direcciones (I)

Segmento heap

Verificar límites

$$\text{offset} = \text{VA} - \text{VA}(\text{heap})$$

$$\text{offset} = 7\text{K} - 4\text{K}$$

$$\text{offset} = 3\text{K}$$

$$0 \leq \text{offset} < \text{size}(\text{heap})$$

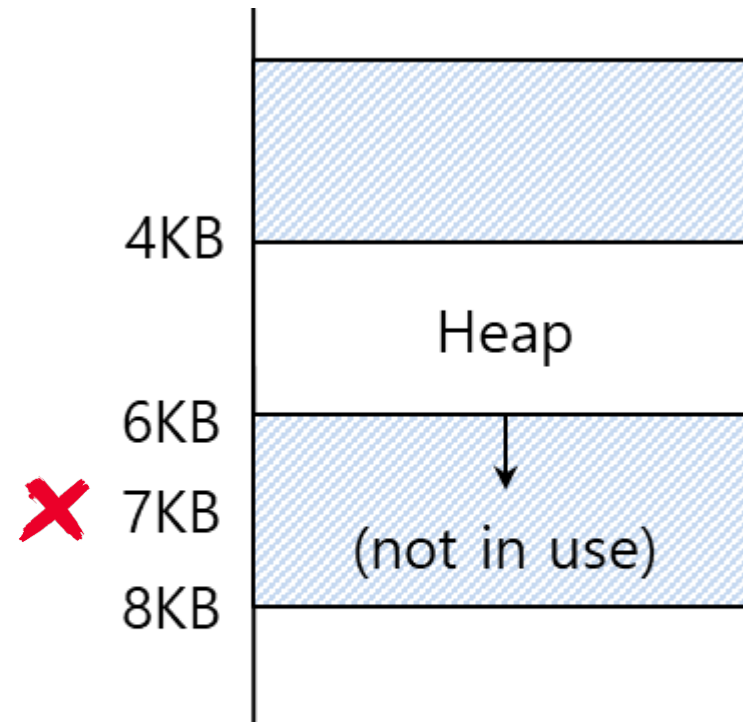
$$0 \leq 3\text{K} < 2\text{K}$$



Como la dirección a la que se hace referencia está fuera de los límites, el Sistema Operativo lanza un **fallo de segmentación**

Pregunta: ¿Cuál es la PA de VA = 7K?

- La dirección se encuentra más allá del heap



Address Space

Referenciando un segmento

Ejemplo:

Dado el espacio de direcciones mostrado en la siguiente figura ¿Cuántos bits son necesarios para direccionar?

- **Datos:**

- **size(VM) = 16K** (tamaño de la memoria virtual)
- **n = ?** (Número de bits necesarios para direccionar)

$$2^n = 16K$$

$$2^n = 16(2^{10})$$

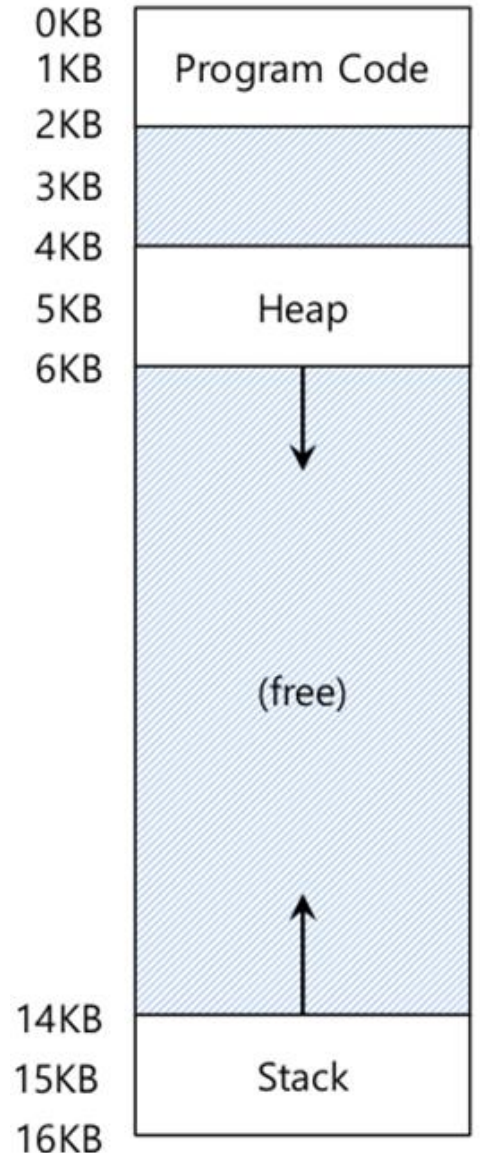
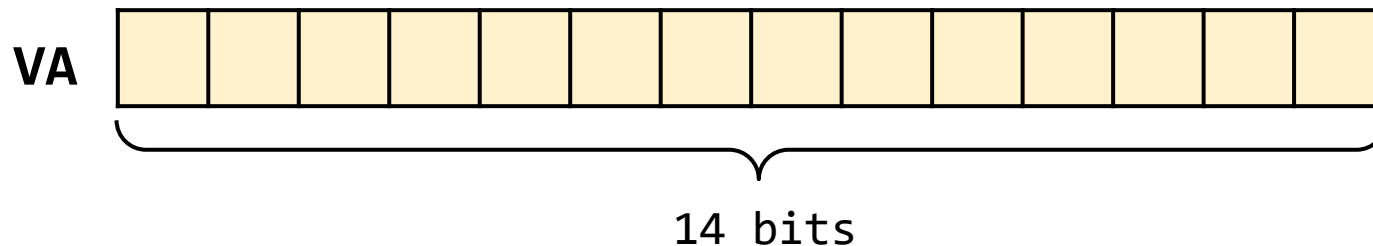
$$2^n = (2^4)(2^{10})$$

$$2^n = 2^{14}$$

$$\log_2(2^n) = \log_2(2^{14})$$

$$n = 14$$

Solución: Se necesitan **14 bits**.

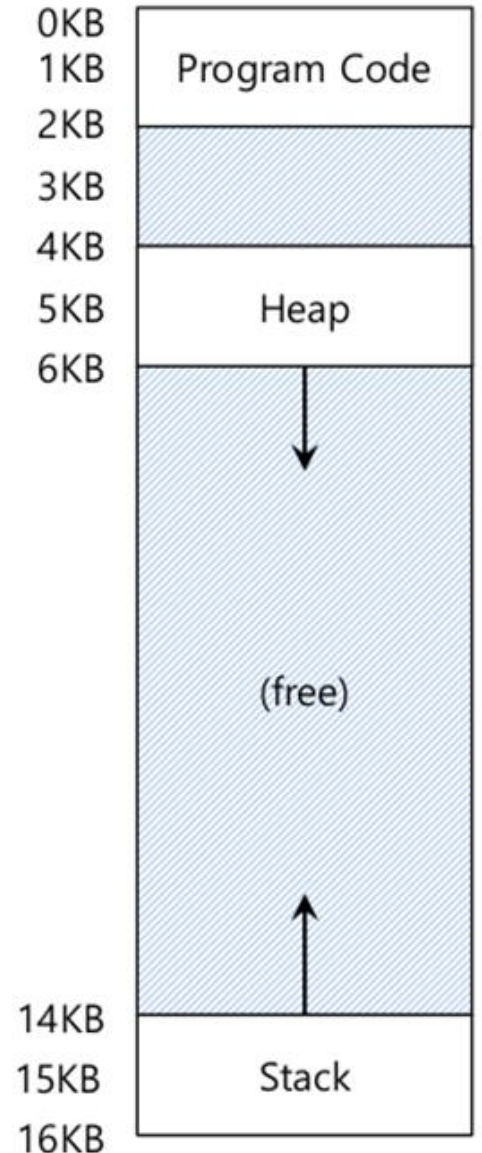
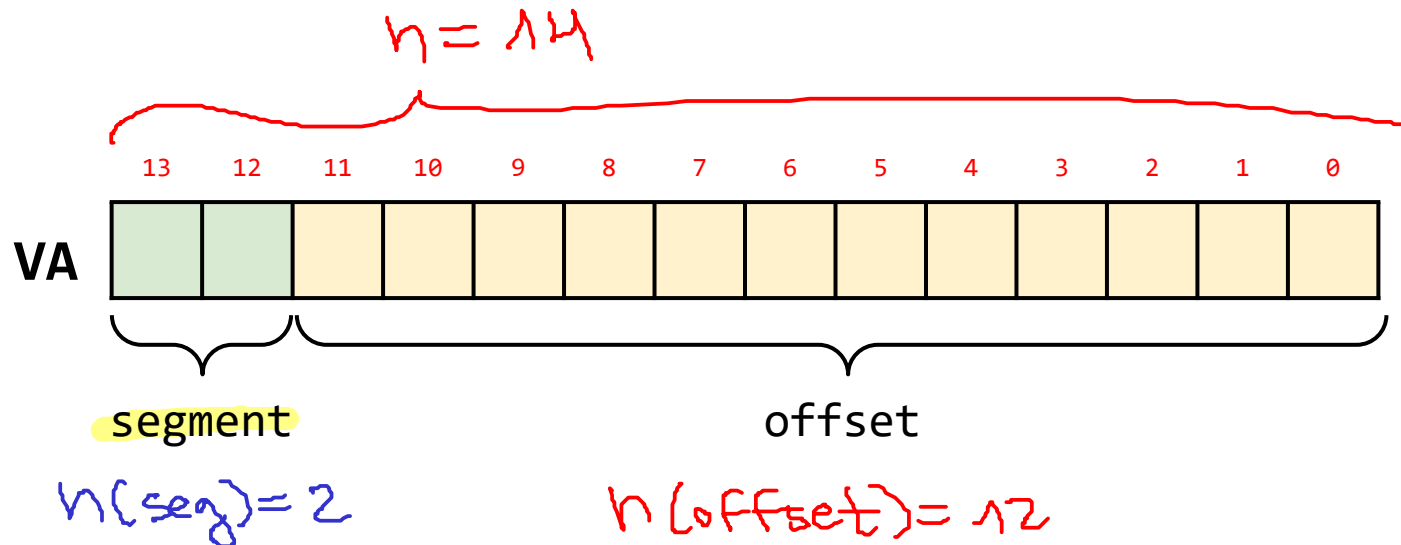


Referenciando un segmento

Ejemplo:

Anteriormente se demostró que con 14 bits se puede direccionar una memoria de 16K. Se tienen 3 segmentos de modo que: **¿Como sabemos a cuál segmento pertenece una dirección determinada?**

segment	bits
code	00
heap	01
stack	11
---	10

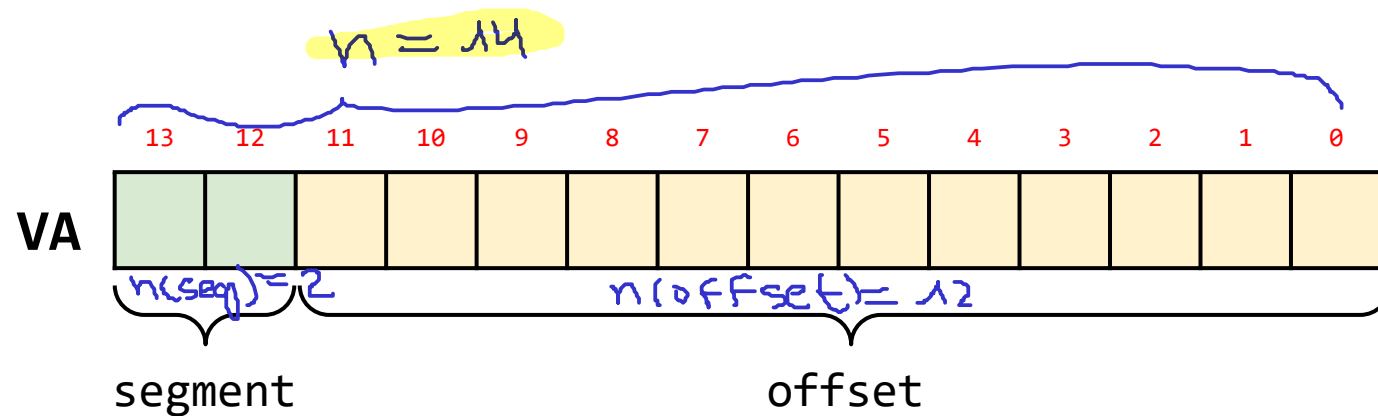


Referenciando un segmento

Explicit approach:

Divide el espacio de direccionamiento de tal manera que una dirección virtual está compuesta por dos partes [**segment** | **offset**]

- **segment**: Compuesto por los bits más significativos (top bits) de la dirección lógica.
- **Offset dentro del segmento elegido**: Parte compuesta por los bits menos significativos que hace referencia al offset respecto al segmento seleccionado.



segment	bits
code	00
heap	01
stack	11
	10

VA = 104: $VA = 104 = 0 \times 68 = 0 \times 0068 = \underline{000} \underline{0000} \underline{1101000}$ = [00 | 000001101000]
VA = 4200: Resuelto en las próximas slides
seg = code = 00
offset = 104

Referenciando un segmento

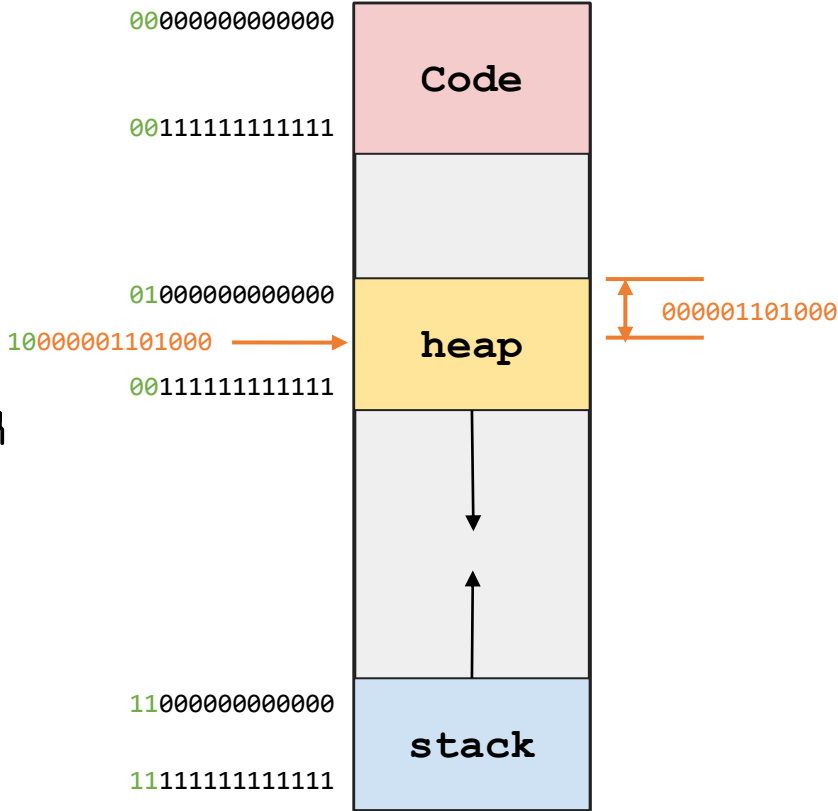
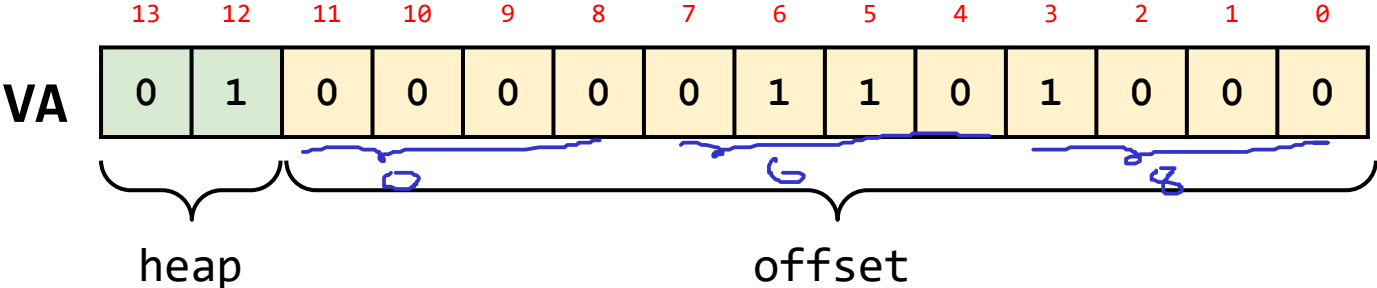
Ejemplo:

Dada la dirección virtual VA = 4200 ¿Cual es el segmento al que pertenece?

VA = 4200 = 0x1068 = ~~0001~~ 0000 0110 1000
h = 14

bits	segment	base	size
00	code	32K	2K
01	heap	<u>34K</u>	2K
11	stack	28K	2K
---	---	---	---

seg = 01 = heap
offset = 0x68 = 104



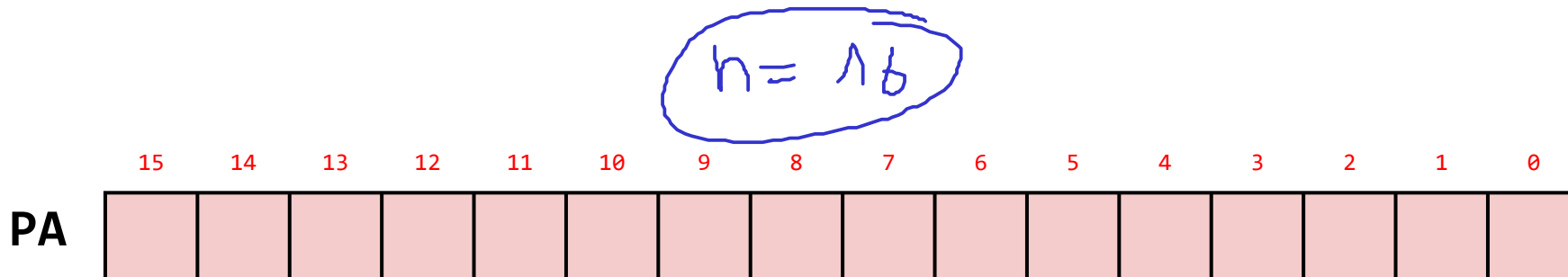
Referenciando un segmento

Ejemplo:

Si se tiene una memoria física de 64K **¿Cuántos bits son necesarios para direccionarla?**

- **Datos:**

- **size(PM) = 64K** (tamaño de la memoria física)
- **n = ?** (Número de bits necesarios para direccionar la memoria física)



$$2^n = 64K$$

$$2^n = 64(2^{10})$$

$$2^n = (2^6)(2^{10})$$

$$2^n = 2^{16}$$

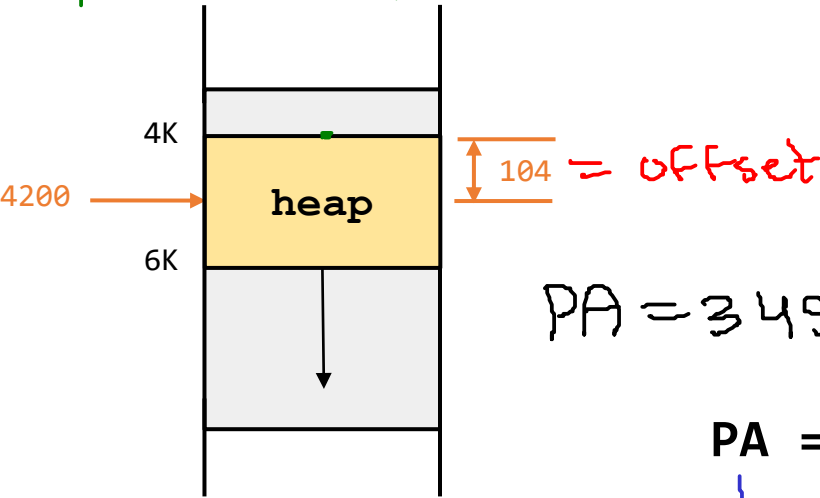
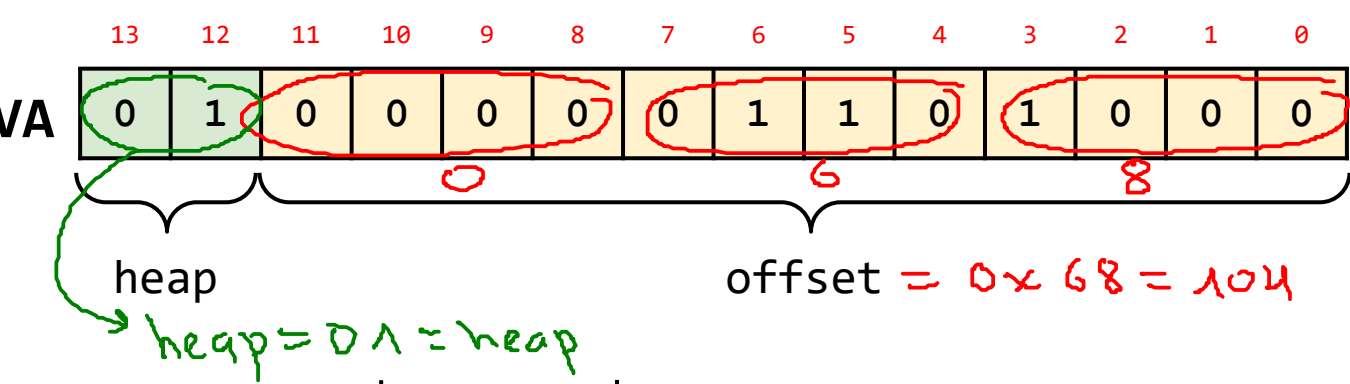
$$\log_2(2^n) = \log_2(2^{16})$$

$$n = 16$$

Referenciando un segmento

Ejemplo:

Teniendo en cuenta lo anterior ¿Cual la dirección física asociada?



VA = 4200 = 01 0000 0110 1000

- segment: heap
- offset: 0000 0110 1000 = 0x068 = 104

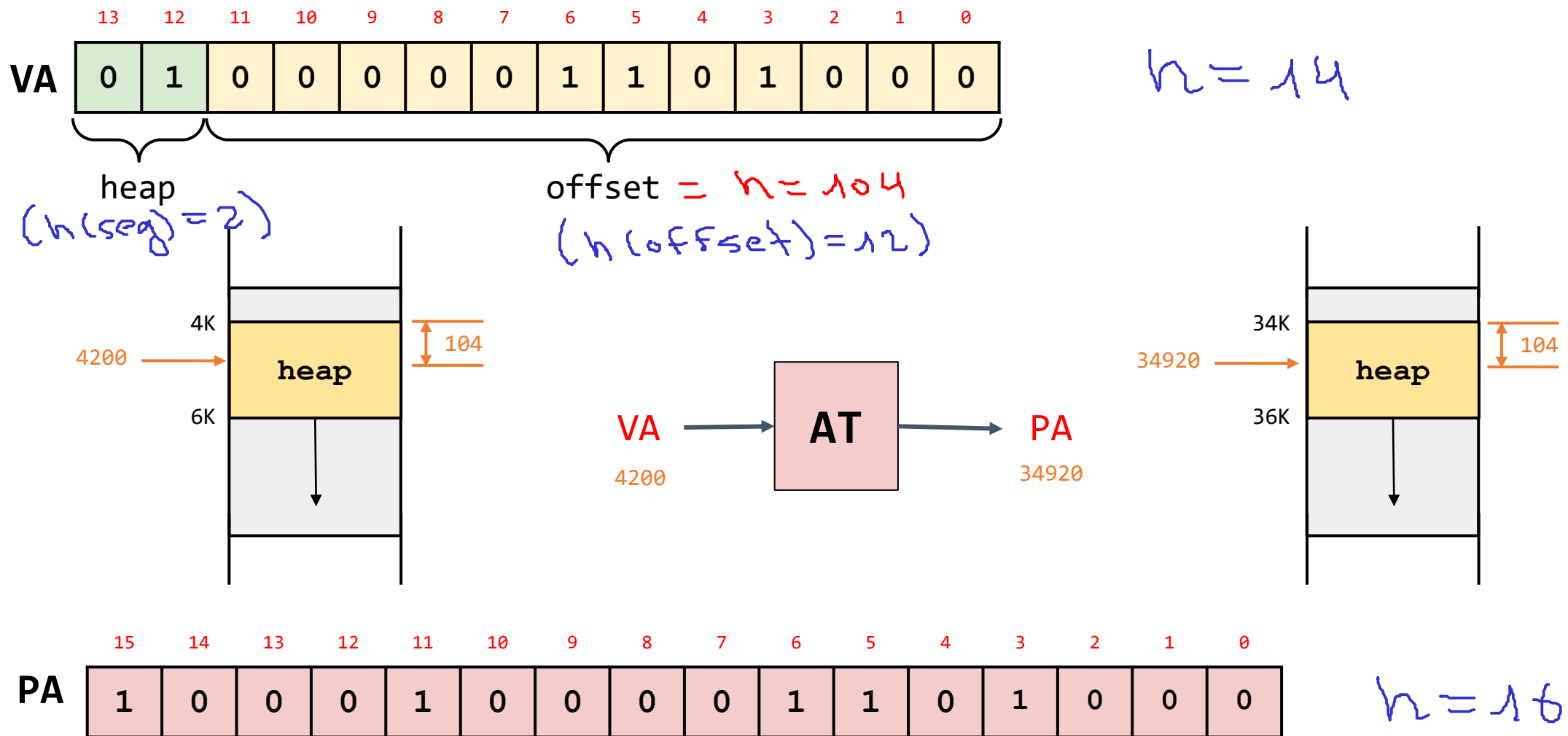
bits	segment	base	size
00	code	32K	2K
01	heap	34K	2K
11	stack	28K	2K
---	---	---	---

PA = 34920 = 0x8868 = [1000 1000 0110 1000]

PA = 34920 = 1000 1000 0110 1000
 ↳ n = 16 bits

Referenciando un segmento

Ejemplo:



Traducción de direcciones - Pseudocódigo

$$VA = 4200 = 0 \times 1068 = \overbrace{01000001101000}^{n=14}$$

```

1  // get top 2 bits of 14-bit VA
2  Segment = (VirtualAddress & SEG_MASK) >> SEG_SHIFT
3  // now get offset
4  Offset = VirtualAddress & OFFSET_MASK
5  if (Offset >= Bounds[Segment])
6      RaiseException(PROTECTION_FAULT)
7  else
8      PhysAddr = Base[Segment] + Offset
9      Register = AccessMemory(PhysAddr)
    
```

#seg = 01

#offset = 0x68 = 104

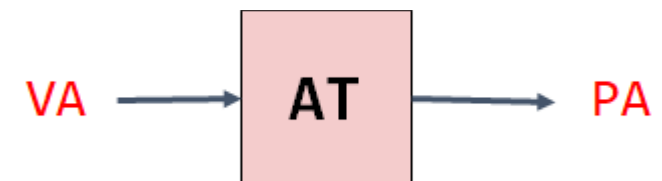
34K

104

bits	segment	base	size
00	code	32K	2K
01	heap	34K	2K
11	stack	28K	2K
---	---	---	---

- SEG_MASK = 0x3000 (11000000000000)
- SEG_SHIFT = 12 (n(offset) = 12)
- OFFSET_MASK = 0xFFF (00111111111111)

13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	0	0	1	1	0	1	0	0	0



15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	1	0	0	0	0	1	1	0	1	0	0	0

Traducción de direcciones (II)

Fin: 10/09/2026
Inicio: 15/09/2026

Segmento stack (pila)

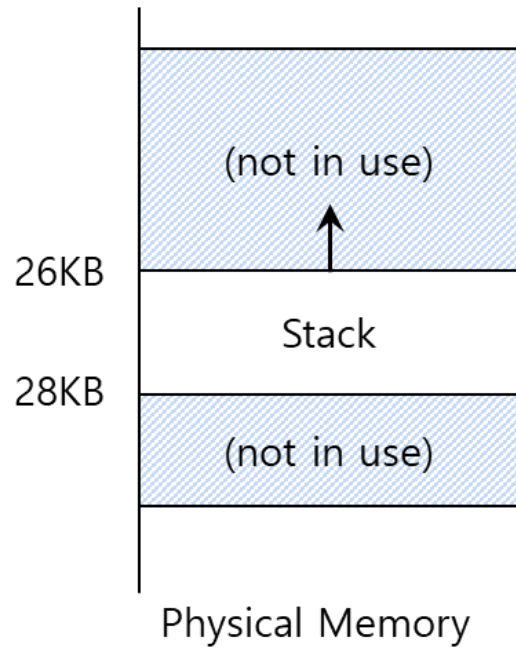
- La pila crece **hacia atras (grows background)**
- Es necesario agregar soporte de hardware adicional que indique el sentido de crecimiento del segmento:
 - **0**: Crecimiento en dirección negativa
 - **1**: Crecimiento en dirección positiva

segment	base	size (max 4K)	Grows Positive?
code(00)	32K	2K	1
heap(01)	34K	2K	1
stack(11)	28K	2K	0

Segment Registers (With Negative-Growth Support)

Traducción de direcciones (II)

Segmento stack (pila)



segment	base	size (max 4K)	Grows Positive?
code(00)	32K	2K	1
heap(01)	34K	2K	1
stack(11)	28K	2K	0

Segment Registers (With Negative-Growth Support)

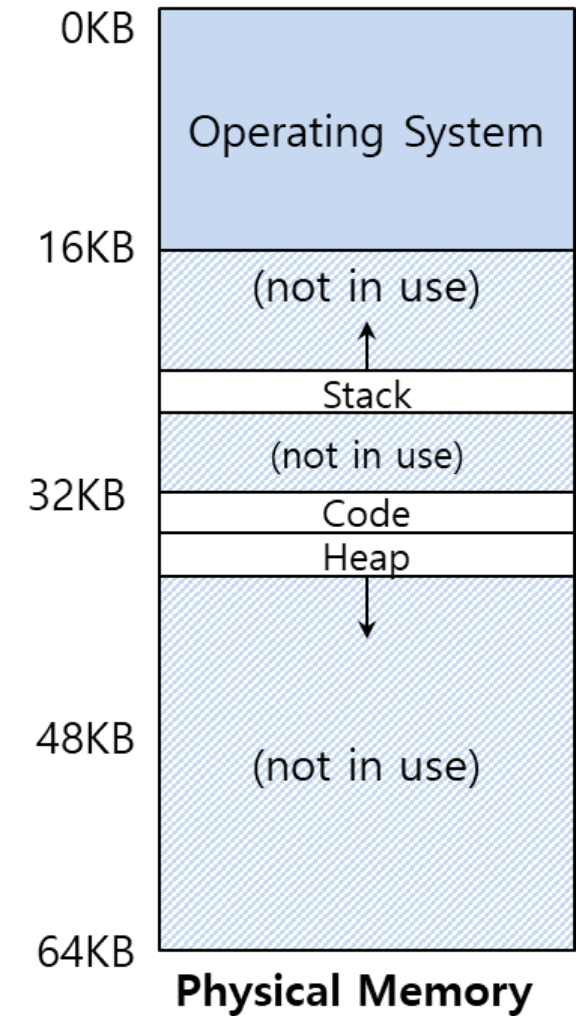
Traducción de direcciones (II)

Ejemplo

Para el mismo ejemplo que se ha venido trabajando y teniendo en cuenta los segmentos mostrados en la tabla adjunta. Si $VA = 15K$, determine:

1. ¿A cuál segmento pertenece dicha VA?
2. ¿Cual es el offset?
3. ¿Cual es la dirección física?

segment	base	size (max 4K)	Grows Positive?
code(00)	32K	2K	1
heap(01)	34K	2K	1
stack(11)	28K	2K	0

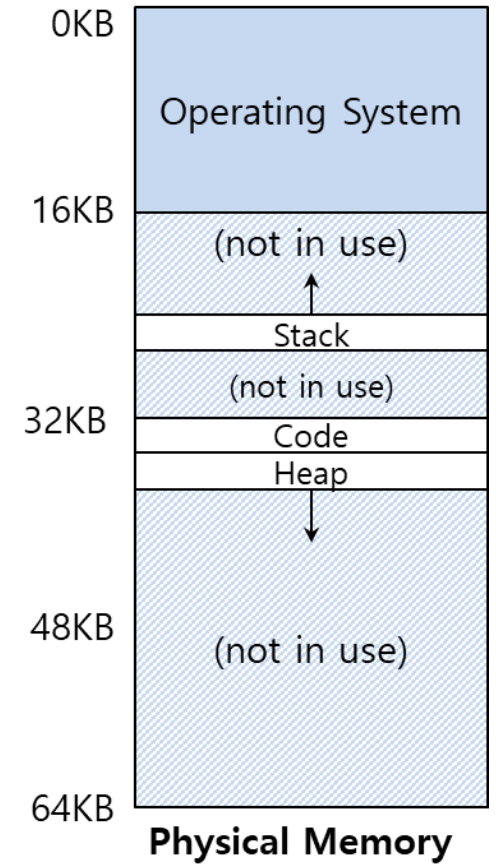
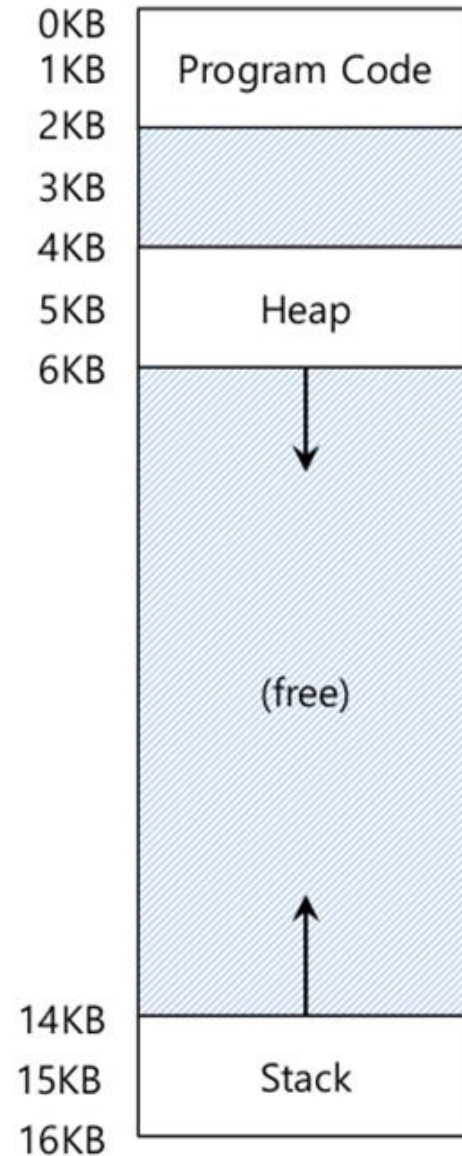


Traducción de direcciones (I)

Pregunta:

- Si $VA = 15K$, ¿cual es el valor de PA ?

segment	base	size
code	32K	2K
heap	34K	2K
stack	28K	2K



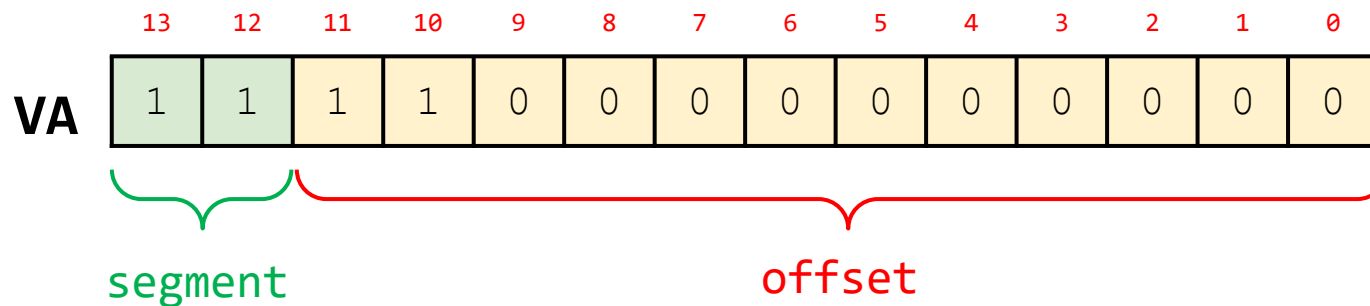
Traducción de direcciones (II)

Datos ejemplo

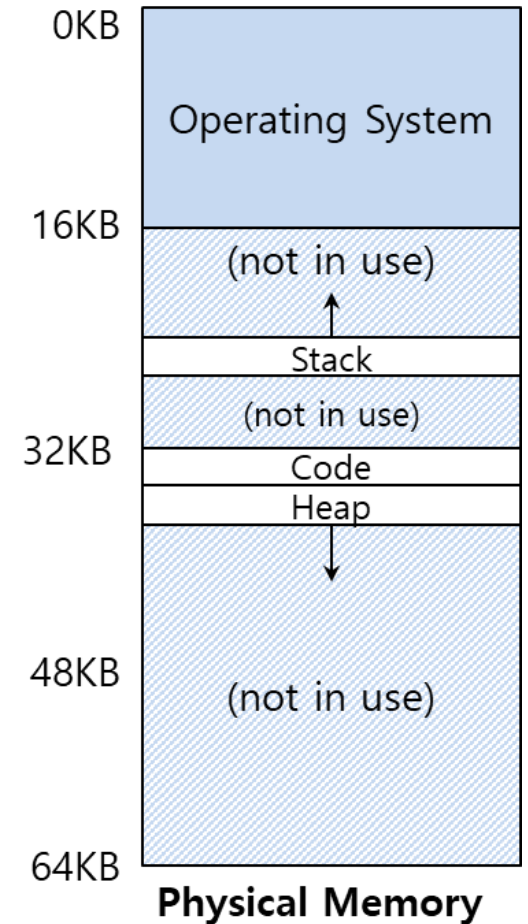
- VA = 15K
- $\text{seg}(\text{VA}) = ?$

segment	base	size (max 4K)	Grows Positive?
code(00)	32K	2K	1
heap(01)	34K	2K	1
stack(11)	28K	2K	0

$$\text{VA} = 15\text{K} = 15(2^{10}) = 15(1024) = 15360 = 0x3C00$$



Segmento: stack offset: _____

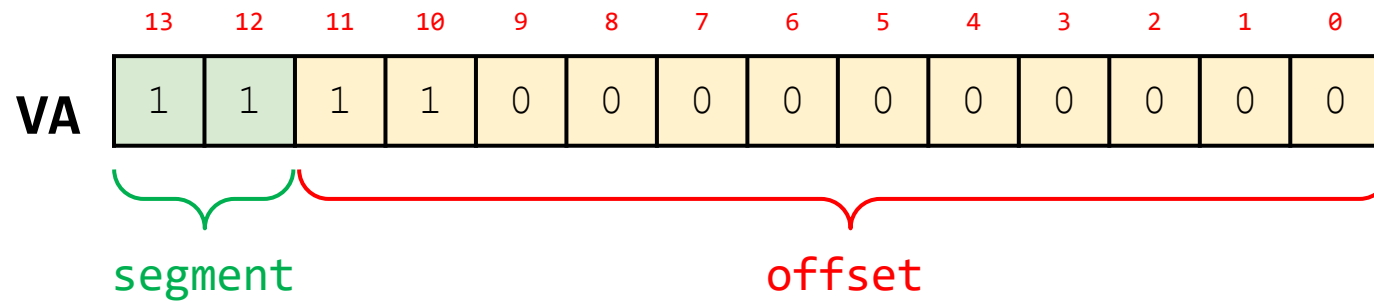


Traducción de direcciones (II)

Datos ejemplo

- VA = 15K
- offset(VA) = ?

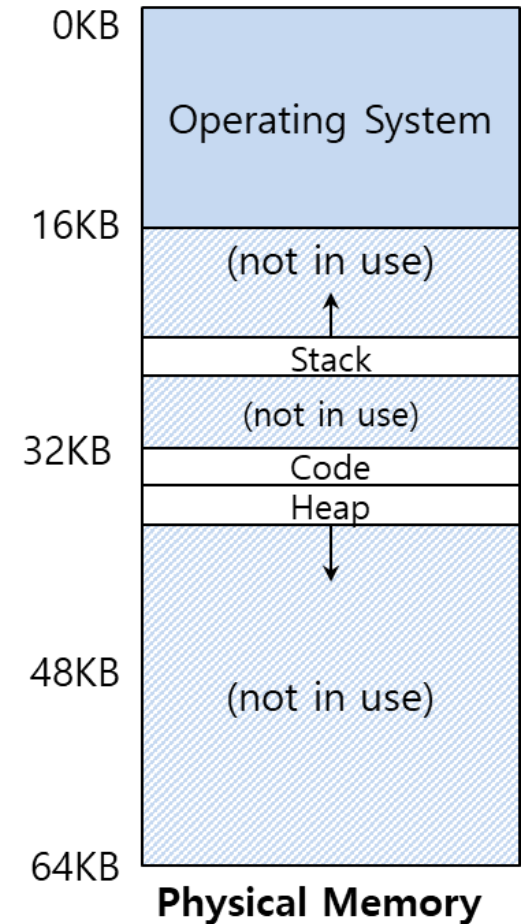
segment	base	size (max 4K)	Grows	Positive?
code(00)	32K	2K		1
heap(01)	34K	2K		1
stack(11)	28K	2K		0



$$\text{offset} = 1100\ 0000\ 0000 = 0xC00 = 3072$$

$$\text{offset} = 3072/1024 = 3K$$

Segmento: stack **offset:** 3K

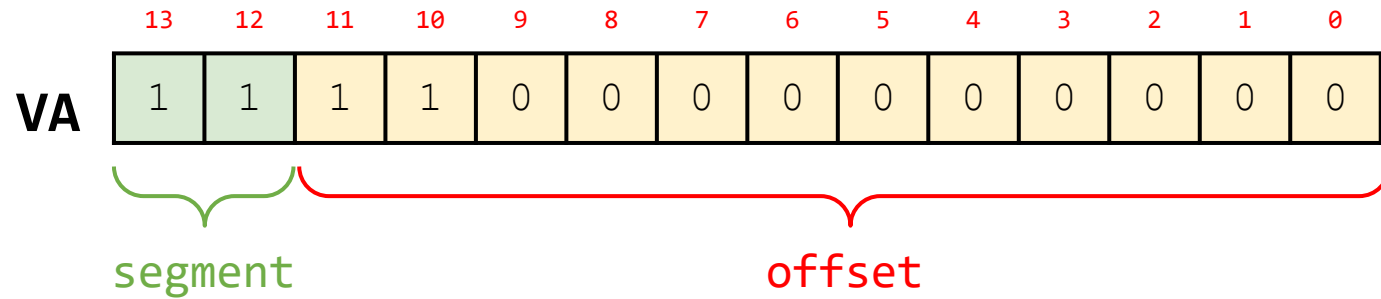


Traducción de direcciones (II)

Datos ejemplo

- VA = 15K
- offset(VA) = ?

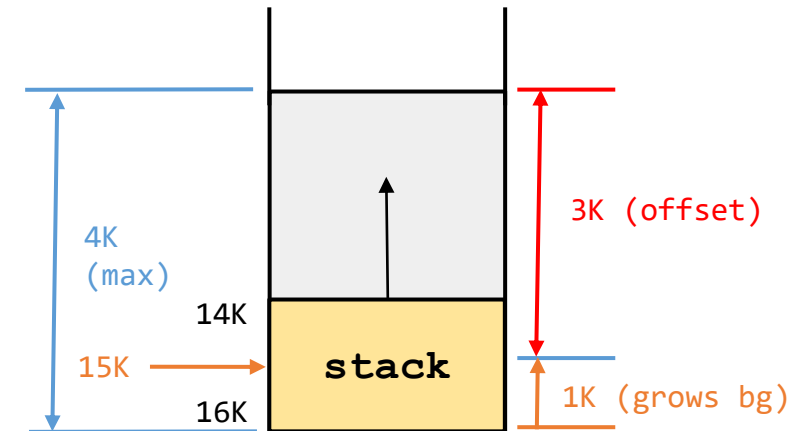
segment	base	size (max 4K)	Grows Positive?
code(00)	32K	2K	1
heap(01)	34K	2K	1
stack(11)	28K	2K	0



- segment = stack
- offset = 3K

Para el offset hay 12 bits, por lo tanto el **offset máximo** será:

- $\text{offset(max)} = 2^{12} = 4096 = 4K \text{ (max)}$



Traducción de direcciones (II)

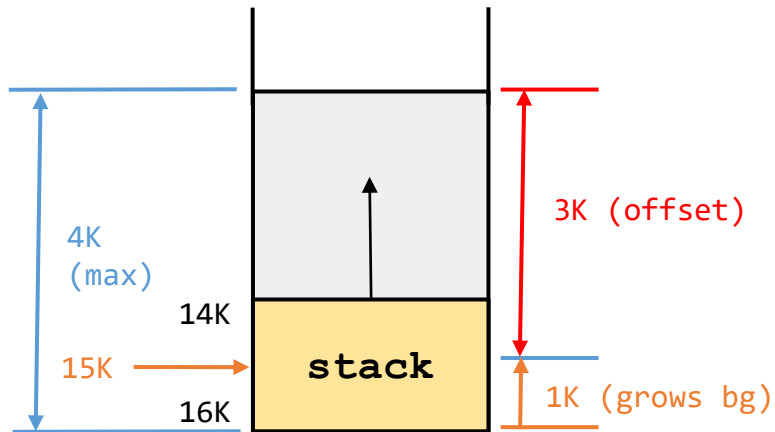
Segmento stack (pila)

- VA = [11|1100000000]
 - segment = stack
 - offset = 3K

$$\text{offset(stack)} = \text{offset} - \text{offset(max)}$$

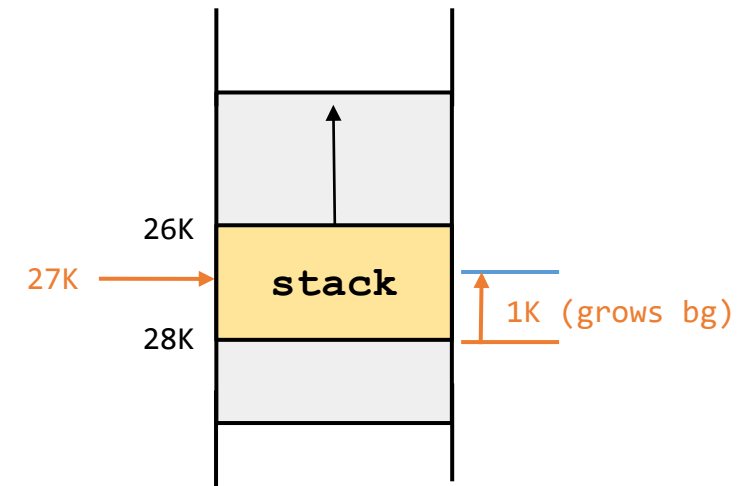
$$\text{PA} = \text{base(stack)} + \text{offset(stack)}$$

Cálculo de la dirección física (PA):



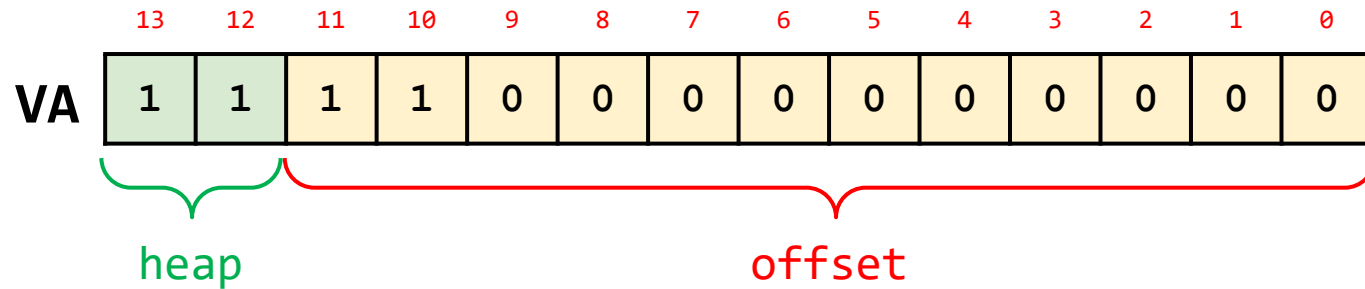
$$\text{offset(stack)} = 3K - 4K = -1K$$

$$\text{PA} = 28 + (-1K) = 27K$$

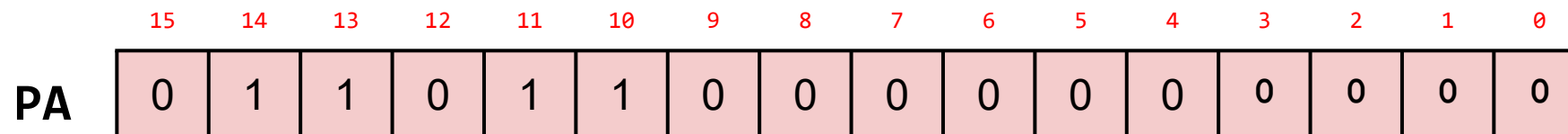
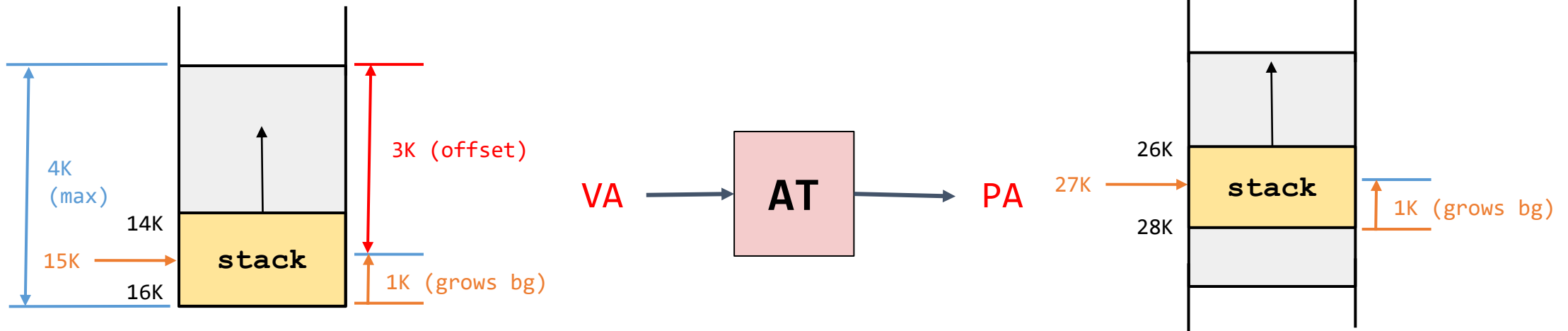


Traducción de direcciones (II)

Ejemplo



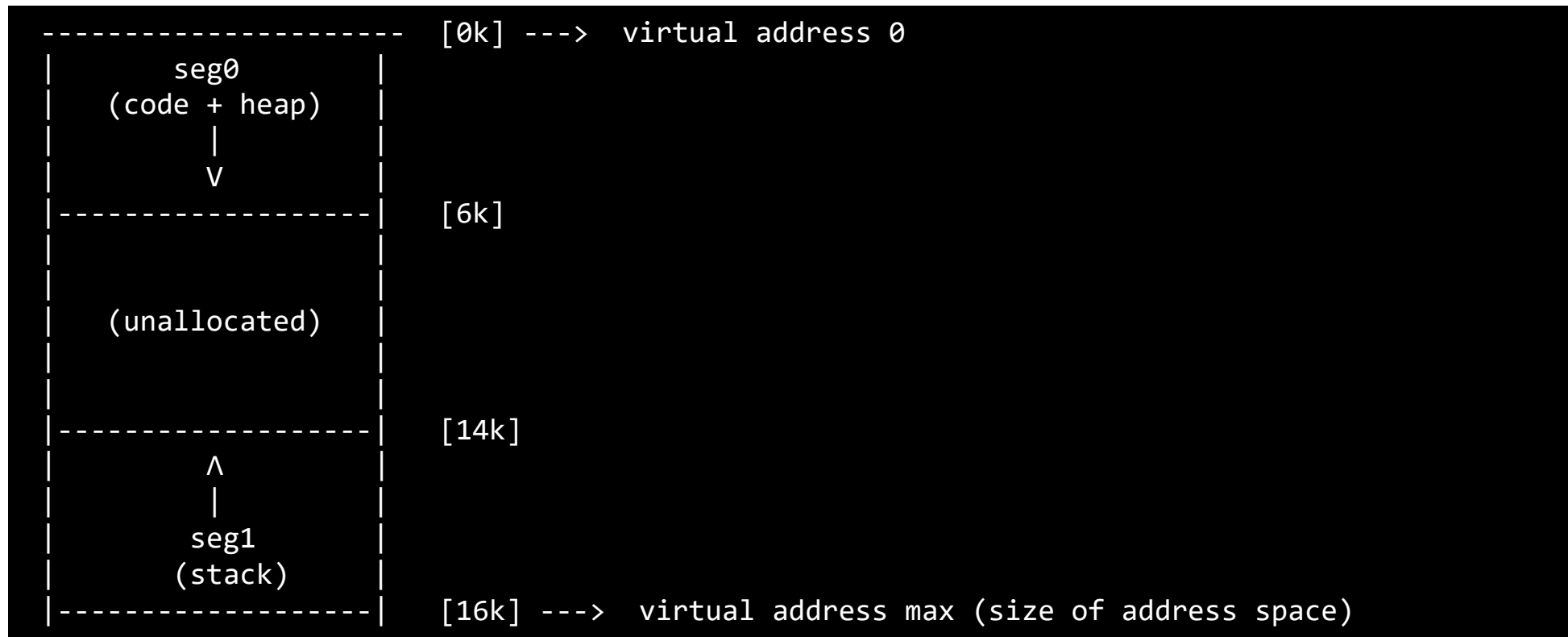
segment	base	size (max 4K)	Grows Positive?
code(00)	32K	2K	1
heap(01)	34K	2K	1
stack(11)	28K	2K	0



Traducción de direcciones - Simulación

Simulador: A continuación se muestra el comando básico para definir los la memoria virtual y la distribución de los segmentos tanto en memoria virtual como en memoria física (las direcciones virtuales se generan en este caso aleatoriamente):

```
./segmentation.py -a 16k -p 64k -b 32k -l 6k -B 28k -L 2k -s 22
```



Traducción de direcciones - Simulación

Simulador: A continuación se muestra el comando básico para definir los la memoria virtual y la distribución de los segmentos tanto en memoria virtual como en memoria física. En este caso se pasan las direcciones virtuales a mapear (100, 4200, 7K y 15K).

```
./segmentation.py -A 100,4200,7096,15360 1 -a 16k -p 64k -b 32k -l 6k -B 28k -L 2k
```

```
ARG seed 0
ARG address space size 16k
ARG phys mem size 64k

Segment register information:

Segment 0 base (grows positive) : 0x00008000 (decimal 32768)
Segment 0 limit                  : 6144

Segment 1 base (grows negative) : 0x00007000 (decimal 28672)
Segment 1 limit                  : 2048

Virtual Address Trace
VA 0: 0x00000064 (decimal: 100) --> PA or segmentation violation?
VA 1: 0x00001068 (decimal: 4200) --> PA or segmentation violation?
VA 2: 0x00001bb8 (decimal: 7096) --> PA or segmentation violation?
VA 3: 0x00003c00 (decimal: 15360) --> PA or segmentation violation?

...
```

Traducción de direcciones - Simulación

Simulador: A continuación se muestra el comando básico para definir los la memoria virtual y la distribución de los segmentos tanto en memoria virtual como en memoria física. En este caso se muestran las direcciones virtuales a mapear (100, 4200, 7K y 15K) y sus respectivas direcciones físicas en los casos donde es posible.

```
./segmentation.py -A 100,4200,7096,15360 1 -a 16k -p 64k -b 32k -l 6k -B 28k -L 2k -c
```

```
ARG seed 0
ARG address space size 16k
ARG phys mem size 64k

Segment register information:

Segment 0 base (grows positive) : 0x00008000 (decimal 32768)
Segment 0 limit                  : 6144

Segment 1 base (grows negative) : 0x00007000 (decimal 28672)
Segment 1 limit                  : 2048

Virtual Address Trace
VA 0: 0x00000064 (decimal: 100) --> VALID in SEG0: 0x00008064 (decimal: 32868)
VA 1: 0x00001068 (decimal: 4200) --> VALID in SEG0: 0x00009068 (decimal: 36968)
VA 2: 0x00001bb8 (decimal: 7096) --> SEGMENTATION VIOLATION (SEG0)
VA 3: 0x00003c00 (decimal: 15360) --> VALID in SEG1: 0x00006c00 (decimal: 27648)

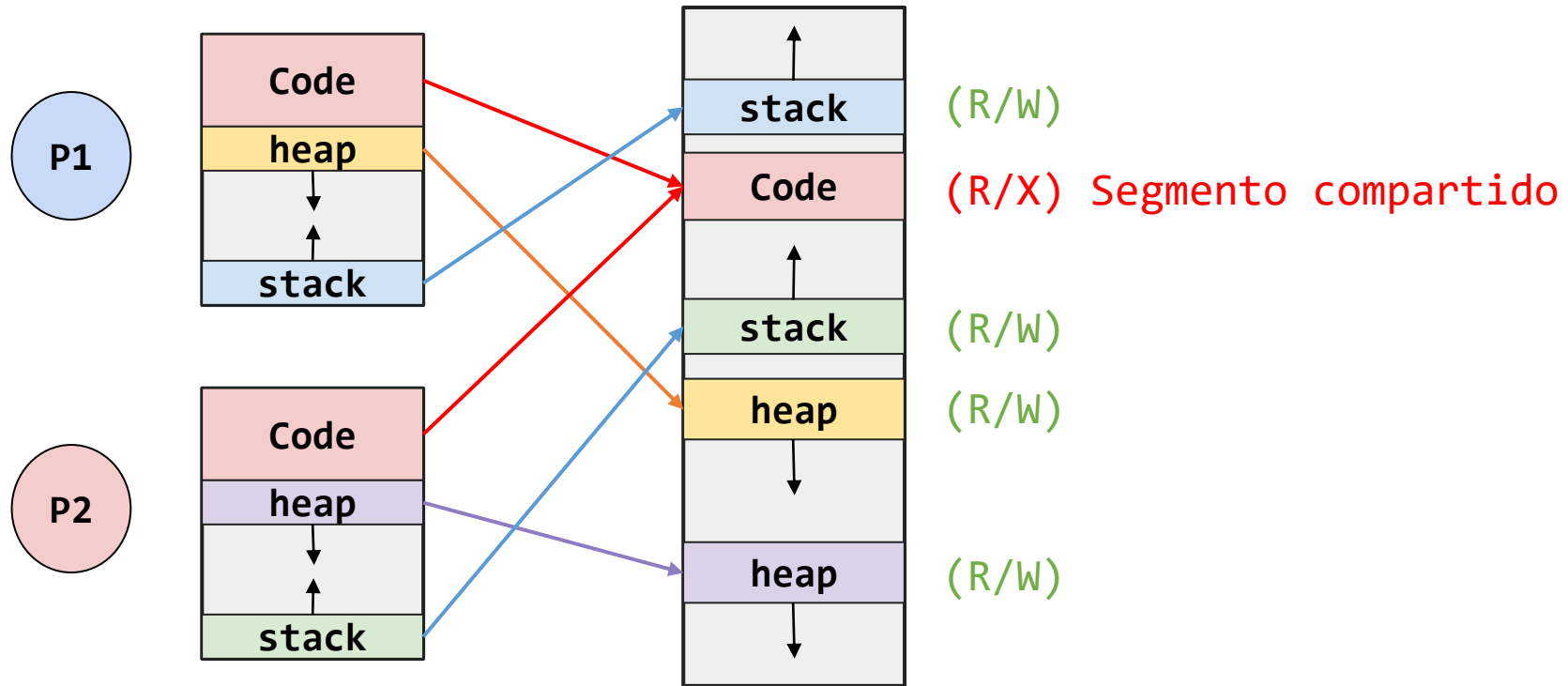
...
```

Segmentos compartidos

Escenario

¿Que pasa cuando se ejecutan dos procesos con la misma imagen?

- Algunos segmentos (como el segmento de código) podrían compartirse

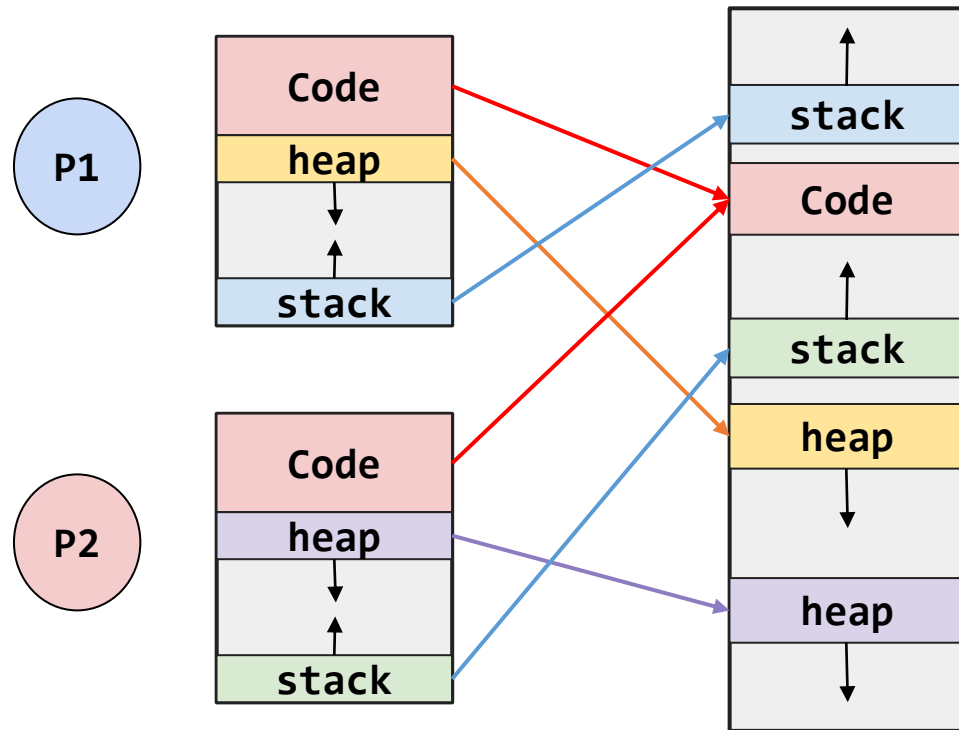


Segmentos compartidos

Escenario

¿Que pasa cuando se ejecutan dos procesos con la misma imagen?

- Algunos segmentos (como el segmento de código) podrían compartirse



¿Cómo hacerlo?

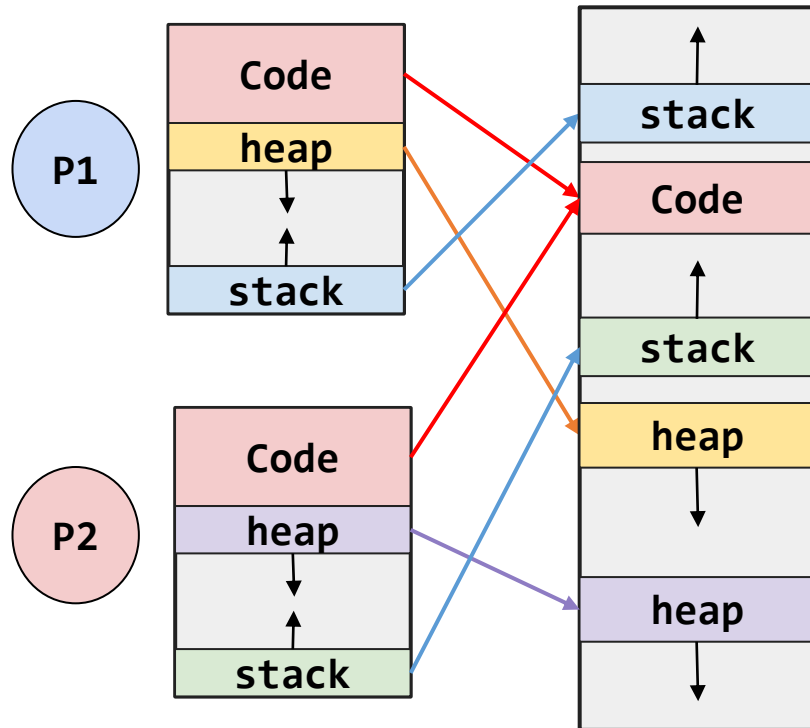
- Se agrega HW adicional usando **bits de protección** para indicar los permisos de un segmento en particular:
 - R**: Lectura
 - W**: Escritura
 - X**: Ejecución

Segmentos compartidos

Escenario

¿Que pasa cuando se ejecutan dos procesos con la misma imagen?

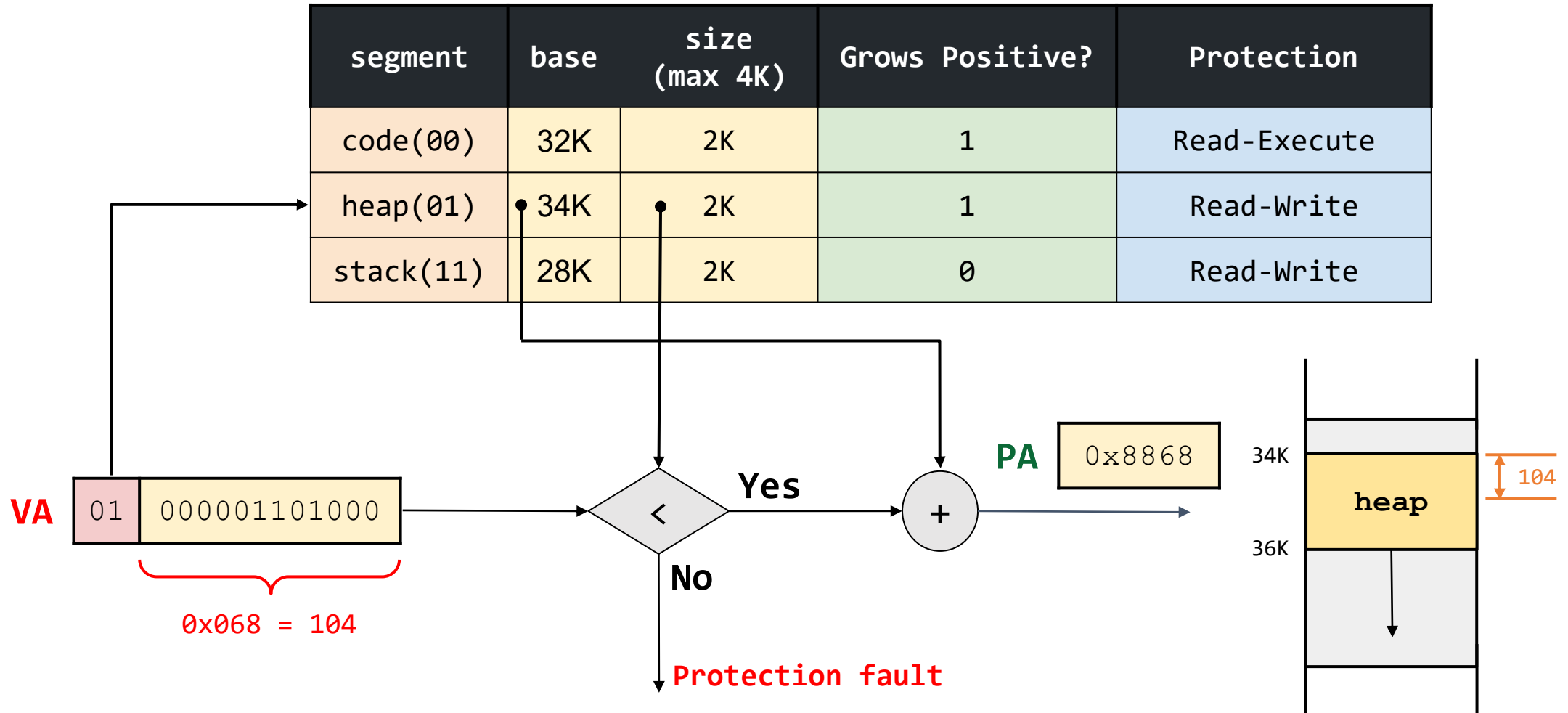
- Algunos segmentos (como el segmento de código) podrían compartirse



segment	base	size (max 4K)	Grows Positive?	Protección
code(00)	32K	2K	1	Read-Execute
heap(01)	34K	2K	1	Read-Write
stack(11)	28K	2K	0	Read-Write

Segment Register Values (with Protection)

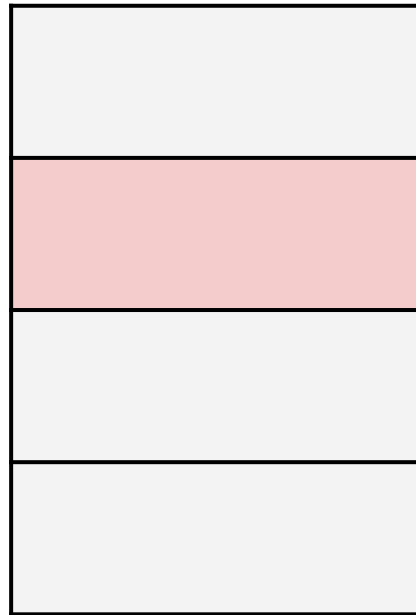
Hardware de segmentación



Segmentation granularity

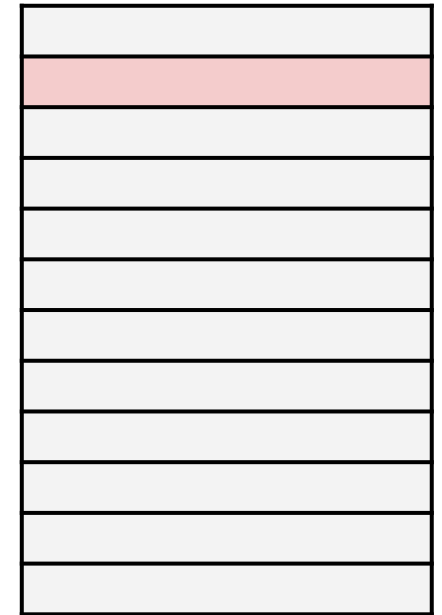
Grano grueso (Coarse-grained)

- Se utilizan pocos segmentos relativamente grandes (ej: code, heap, stack)



Grano fino (Fined-grained)

- Emplea muchos segmentos pequeños
 - Mayor flexibilidad.
 - Requiere más soporte de hw (Tabla de segmentos).



Ventajas y desventajas de la segmentación

Ventajas

- Permite espacios de direcciones pequeños. El stack y el heap pueden crecer independientemente.
- Ahorro de memoria al compartir segmentos.
- Facilita la protección de los segmentos.

Desventajas

- **Fragmentación externa:** Pequeños huecos de espacio libre en memoria física difíciles de utilizar

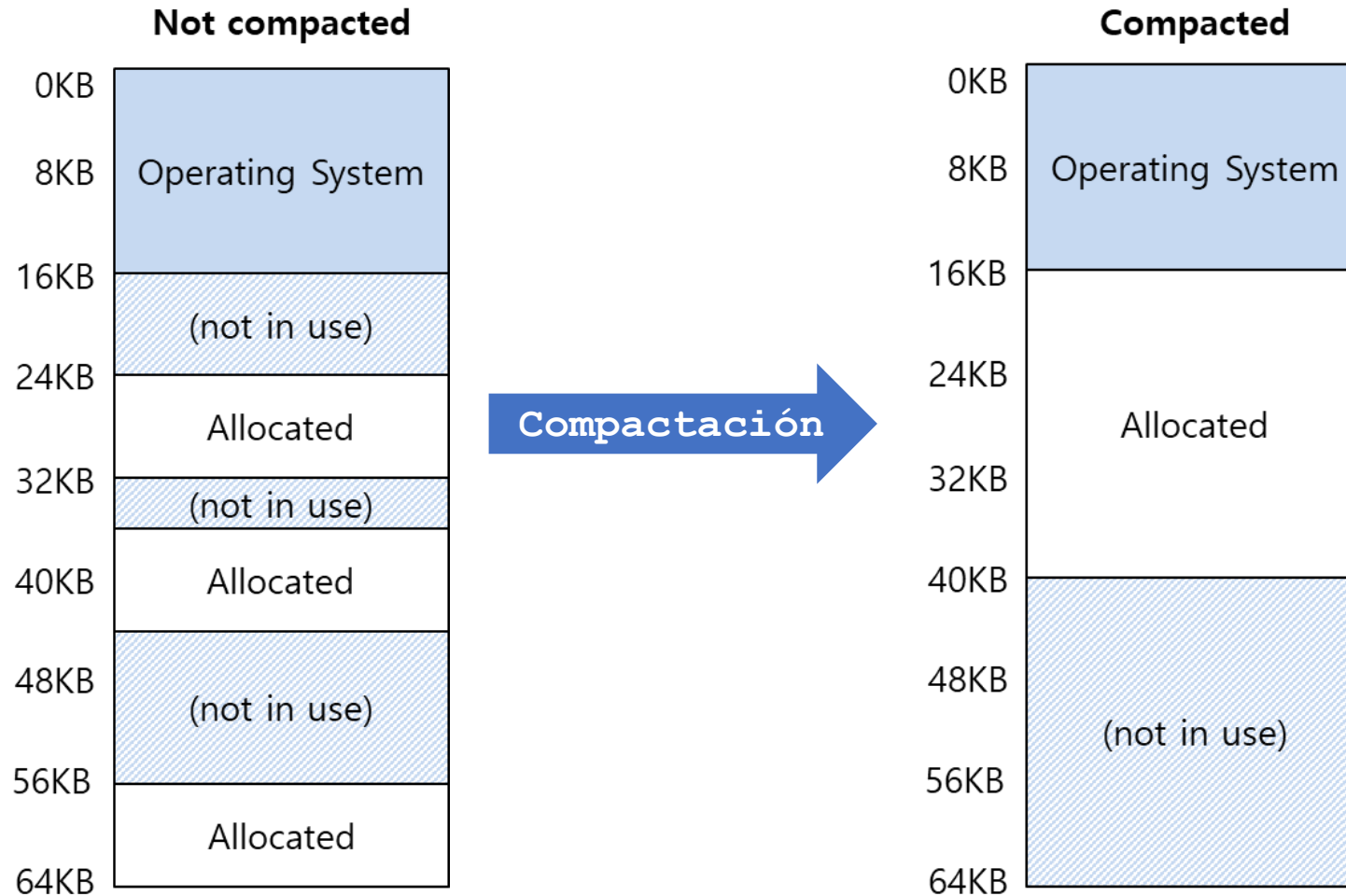
Ejemplo: Dada la siguiente memoria física, responder:

- ¿Cuanta memoria memoria tiene ocupada?
Rta: 40K
- ¿Cuanta memoria libre?
Rta: 24K, pero no están contiguos los segmentos.
- ¿Es posible ubicar un segmento con un tamaño de 20K?
Rta: No es posible

8KB	Operating System
16KB	(not in use)
24KB	Allocated
32KB	(not in use)
40KB	Allocated
48KB	(not in use)
56KB	Allocated
64KB	Allocated

Compactación

Reorganización de los **segmentos** existentes en memoria física



Proceso Costoso:

- Detener procesos en ejecución
- Copiar datos en un nuevo lugar
- Cambiar valores de los registro para el segmento

Algoritmos de ubicación:

- Best-fit
- First-fit
- Worse-fit

Referencias

- **Operating Systems Three Easy Pieces** ([website](#): Capítulo [16](#))
- Videos de youtube: Clase 7- Segmentación ([link](#)).
- Página de Remzi H. Arpaci-Dusseau ([link](#))
- Operating System Concepts - Tenth Edition ([website](#))
- Operating Systems Notes ([link](#))
- <https://github.com/Aniruddha-Tapas/Operating-Systems-Notes/blob/master/1-Overview.md>
- <https://myaut.github.io/dtrace-stap-book/kernel/proc.html>
- <https://myaut.github.io/dtrace-stap-book/index.html>
- <https://www.technologyuk.net/computing/computer-software/operating-systems/operating-system-utilities.shtml>
- <http://www.it.uu.se/education/course/homepage/os/vt18/module-4/simple-threads/>
- <https://courses.cs.duke.edu/spring19/compsci590.1/slides/>
- <http://csl.snu.ac.kr/courses/4190.307/2020-1/>
- <https://fog.misty.com/perry/>
- <https://fog.misty.com/perry/os/>

